// QE LEARNING PATH · PHASE P02 · ~23 H

Python for QE

Seven tasks that take you from lists and dicts to typed, packaged framework code — the Python that test frameworks are built from, and the Python coding round. Every example from **P02-python.md** is here: **run it live** on the page, **watch it animate** where that helps, then answer the interview questions out loud.

- 🧠 **Level 1 — Crack It** · the 🟢 low example
- 🚀 **Level 2 — Impress Them** · the 🔵 medium example
- 🏆 **Level 3 — Hero Mode** · the 🟣 high/complex example
- 🟢 Zero 🔵 Hero 🟣 Super Hero task levels
- Z H SH interview-question levels
- + ADDED not in P02-python.md — added to fill a gap

P02 — THE PHASE AT A GLANCE

🗺️ Seven tasks, ~23 hours

Goal: write clean, typed Python that test frameworks are built from, and pass the Python coding round. **Apps:** SauceDemo and Toolshop *data* — no browser yet. **Prerequisites:** P00 (P01 recommended).

ID	TASK	LEVEL	TIME	YOU DELIVER
P02-01	**Data types, collections and comprehensions**	Zero	3h	`python_basics/products.py`
P02-02	**Functions, type hints and exceptions**	Zero	3h	`python_basics/checkout.py`
P02-03	**Files and test-data formats**	Zero	2h	`framework/data/saucedemo_users.cs` `python_basics/data_io.py`
P02-04	**OOP and dataclasses**	Hero	3h	`framework/models.py`, `python_basics/cart.py`
P02-05	**Decorators, context managers, logging and config**	Hero	4h	`framework/utils/retry.py`, `wait.py` `framework/config.py`, `.env.example`
P02-06	**Coding interview drills**	Hero	4h	`python_basics/drills.py`
P02-07	**Advanced Python for framework authors**	Super Hero	4h	`framework/api/pagination.py`, `framework/types.py`, `mypy --strict` passing

HOW YOU CHECK YOUR WORK IN P02

Use plain `assert` statements under `if __name__ == "__main__":`. In **P03-01** you convert them into pytest tests. That is why most ▶ Run buttons below print *nothing* on success — a silent run means every assert passed.

HOW TO USE THIS PAGE

Tick the **Done when** boxes as you finish each task (they are saved in this browser, and the contents rail shows your count). Press ✎ **Edit** on any runnable example to change it and run it again — break it on purpose, add a `print()`, try an edge case.

PRIMERS — FUNDAMENTALS

📚 The objects everything is built from

Before the tasks, meet the four things P02 keeps reaching for: **collections**, **names and the objects they point at**, **numbers that are money**, and **strings**. Twenty minutes here saves hours of confusion later.

[ ] The four collections

Python gives you four built-in containers. Choosing the right one is most of the skill: a **list** for things in order, a **tuple** for a fixed record, a **dict** for lookups by key, a **set** for "is it in there?".

TYPE	ORDERED	MUTABLE	DUPLICATES	TYPICAL QA USE
`list`	✅	✅	✅	Rows from a table, test steps
`tuple`	✅	❌	✅	Fixed records, dict keys, parametrize rows
`dict`	✅ (insertion)	✅	Keys unique	JSON payloads, lookups by ID
`set`	❌	✅	❌	Dedup, membership, "what's missing?"

```python
row     = ["standard_user", "secret_sauce"]          # list  - ordered, mutable
creds   = ("standard_user", "secret_sauce")          # tuple - ordered, immutable, hashable
booking = {"firstname": "Ada", "totalprice": 150}    # dict  - key → value, insertion-ordered
seen    = {101, 102, 103}                            # set   - unique, unordered

expected, actual = {1, 2, 3, 4}, {2, 3, 5}
print("missing:", expected - actual)     # {1, 4}
print("extra:  ", actual - expected)     # {5}
print("in both:", expected & actual)     # {2, 3}
```

cost What each operation costs + ADDED

The collections table above is from P02-01. This cost table is the part interviewers actually probe.

OPERATION	BIG-O	WHAT IS HAPPENING
`x in my_list`	`O(n)`	checks every element until it finds one
`x in my_set` · `key in my_dict`	`O(1) average`	hashes `x` and jumps straight to its bucket
`my_list[i]` · `my_dict[key]`	`O(1)`	direct index / hash lookup
`my_list.append(x)` · `my_set.add(x)`	`O(1) average`	add at the end / into a bucket
`my_list.insert(0, x)` · `my_list.pop(0)`	`O(n)`	shifts every other element — use `collections.deque` for queues
`sorted(items)` · `my_list.sort()`	`O(n log n)`	a good comparison sort
`expected - actual` (sets)	`O(len(expected))`	one hash lookup per element — replaces a nested loop

pythonic Idioms P02 uses again and again

`[expr for x in items if cond]`

A **comprehension**: map and filter in one readable line. Also `{…}` for sets and dicts.

`sorted(items, key=…, reverse=…)`

Returns a **new** list. `list.sort()` sorts in place and returns `None`.

`zip(values, values[1:])`

Pairs each element with its neighbour — the idiom behind every "is it sorted?" check.

`Counter(items)`

Counts everything in one pass. `defaultdict(list)` groups the same way.

`expected - actual`

Set difference says "what's missing?" in three characters. `&` = both, `|` = either.

`dict(payload)` / `copy.deepcopy(payload)`

Copy before you change shared data. Shallow copies still share nested objects.

```
prices = [29.99, 9.99, 15.99]
result = prices.sort()            # sorts IN PLACE...
print(result)                     # None   ← the classic bug
print(prices)                     # [9.99, 15.99, 29.99] - the original was changed

names = ["Onesie", "Backpack"]
print(sorted(names), names)       # a NEW sorted list; the original is untouched
```

⚠ GOTCHAS

1. `x = my_list.sort()` makes `x` `None` — a classic bug. **2.** `b = a` is a second **name**, not a copy (next primer). **3.** Sets have no order — never assert on the order you iterate one. **4.** Only hashable things (strings, numbers, tuples, frozen dataclasses) can be dict keys or set members; a list cannot.

→ Names point at objects

A variable in Python is a **name tag** stuck on an object, not a box that holds a value. `b = a` puts a second tag on the **same** object. For a mutable object (list, dict, set) that means a change made through one name is visible through the other — and tests that share a "default payload" dict leak state into each other.

INTERACTIVE — ONE PAYLOAD, TWO NAMES

```
base = {"lastname": "User",
        "bookingdates": {"checkin": "2026-01-01"}}
payload = base
payload["lastname"] = "Smith"
```

base → dict #1 | payload → dict #1

dict #1
"lastname": "Smith"
"bookingdates": → dict #2

dict #2
"checkin": "2026-01-01"

Changed through **payload** — and **base** sees it too. Every test that shares base now sends Smith.

The same three behaviours in real Python:

```python
import copy

a = [1, 2]
b = a                       # same list, two names
b.append(3)
print(a)                    # [1, 2, 3]   ← a changed too!

base = {"firstname": "Ada", "bookingdates": {"checkin": "2026-01-01"}}
shallow = dict(base)                        # outer dict copied, nested dict SHARED
shallow["bookingdates"]["checkin"] = "2026-02-02"
print(base["bookingdates"])                 # {'checkin': '2026-02-02'}   ← leaked into base

deep = copy.deepcopy(base)                  # everything copied, recursively
deep["bookingdates"]["checkin"] = "2026-03-03"
print(base["bookingdates"])                 # still 2026-02-02 - base is safe this time
```

⚠ WHY QE CARES

"Passes alone, fails in the suite" is very often shared mutable data. Copy first (`dict(payload)` , `copy.deepcopy`) or — better — build fresh data per test with a factory function.

$ Numbers that are money

A **float** is a binary approximation: most decimal fractions cannot be stored exactly, so totals drift by fractions of a cent and equality assertions fail. **Decimal**, built from a *string*, stores exactly what you typed.

int — exact, any size

float — fast, approximate

Decimal("29.99") — exact decimal, for prices and totals

quantize + ROUND_HALF_UP — rounding as a stated business rule

```
from decimal import Decimal, ROUND_HALF_UP

print(0.1 + 0.2)                          # 0.30000000000000004
print(Decimal("0.1") + Decimal("0.2"))    # 0.3
print(Decimal(2.675))                     # what the float 2.675 REALLY holds
print(round(2.5), round(3.5))             # 2 4   - banker's rounding (half to even)
print(round(2.675, 2))                    # 2.67  - because of the line above
print(Decimal("2.675").quantize(Decimal("0.01"), rounding=ROUND_HALF_UP))   # 2.68
```

⚠ GOTCHAS

1. `Decimal(29.99)` (from a float) inherits the float's error — always `Decimal("29.99")`. **2.** `round(2.5) == 2`: Python rounds half to **even** (banker's rounding). **3.** A one-cent mismatch between your expected value and the product is a real bug class — confirm which rounding rule the product uses.

" " Strings + ADDED

P02 leans on strings everywhere — error messages, CSV cells, half of the drills — but never introduces them. A **string** is an **immutable** sequence of characters: every "edit" builds a new string.

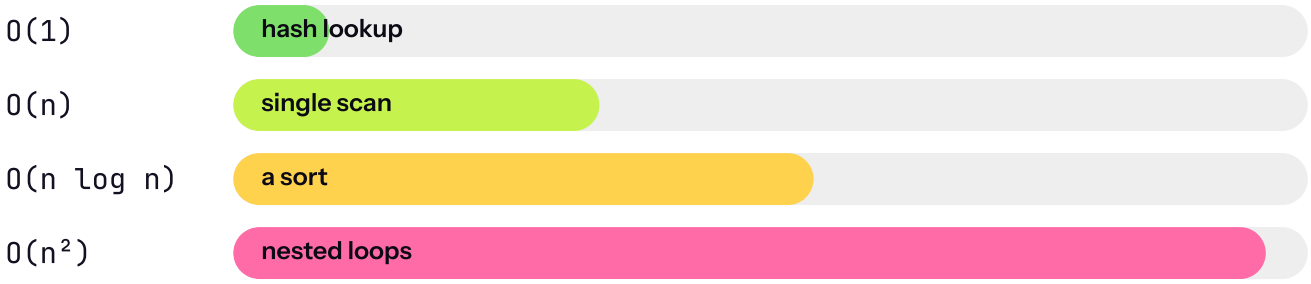
YOU WANT TO…	USE	NOTE
trim whitespace	`s.strip()`	also `lstrip` / `rstrip`; essential on CSV cells and UI text

YOU WANT TO…	USE	NOTE
split / join	`s.split(",")` · `", ".join(parts)`	`join` once is O(n); `+=` in a loop can be O(n²)
compare ignoring case	`a.casefold() == b.casefold()`	like `lower()` but correct for unicode
test content	`"Error" in s` · `s.startswith("Error")`	substring and prefix checks
slice	`s[1:4]` · `s[::-1]`	`stop` is never included; `[::-1]` reverses
format	`f"{price:.2f}"` · `f"{value!r}"`	`!r` shows quotes — it exposes trailing spaces in assertion messages
classify characters	`ch.isalnum()` · `ch.isdigit()`	the palindrome and anagram drills need these

```
s = "  Sauce Labs Backpack  "
print(s.strip().lower())                    # 'sauce labs backpack'
print(s.split())                            # ['Sauce', 'Labs', 'Backpack']
print("-".join(["2026", "01", "05"]))       # 2026-01-05
print(s.strip()[::-1])                      # reversed with a slice
print("Error" in "Error: First Name is required", s.strip().startswith("Sauce"))

price = 29.99
print(f"{'Backpack':<12}|{price:>8.2f}|")    # aligned columns for a report
print(f"{s!r}")                             # !r shows the quotes → hidden whitespace is
visible
print("Straße".casefold() == "STRASSE".casefold())   # True - unicode-safe comparison
```

COMPLEXITY LADDER — THE FOUR RUNGS P02 ASKS YOU TO NAME



With 10k IDs to reconcile, `x in my_list` inside a loop is the difference between **seconds** and — with a set — **milliseconds**. In P02-06 you state the rung for every drill you solve.

TOOLKIT — TECHNIQUES

🧰 Fourteen ideas you'll reuse

Every task below is one or two of these ideas applied to a real QE problem. The first nine build a framework; the last five are the **patterns** P02-06 names for the coding round.

🧺

Comprehensions

Map and filter in one readable line.

When: you are turning one collection into another — names under $20, IDs from a response.

```
cheap = [p["name"] for p in PRODUCTS
         if p["price"] <
Decimal("20")]
```

POWERS → **P02-01 · drills 16, 20**

🔑

sorted(key=...)

A new sorted list, by any rule you can write as a function.

When: mirroring a UI sort dropdown, or ordering records before comparing them.

```
sorted(products, key=lambda p:
p["price"],
       reverse=True)          # a NEW
list
```

POWERS → **P02-01 · drill 13 · P05 sort checks**

🤝

Neighbour pairs

`zip(values, values[1:])` — each element next to the one after it.

When: "is it sorted?", "where is the first violation?" — check the *property*, not a hard-coded list.

```
all(a ⩽ b
    for a, b in zip(values,
values[1:]))
```

POWERS → **P02-01 · reused in P05**

🧮

Set arithmetic

"Missing" and "extra" as one-line expressions.

When: reconciling expected vs actual IDs, or rejecting unknown fields and columns.

```
missing = expected_ids - actual_ids
extra   = actual_ids - expected_ids
```

POWERS → **P02-01 · P02-02 · P02-03**

🪨

Defaults + overrides

A builder returns a valid payload; a test changes only what it cares about.

When: any test data. It is the core of good test-data design.

```
payload =
copy.deepcopy(DEFAULT_BOOKING)
build_booking(lastname="Smith")   #
the rest: defaults
```

POWERS → **P02-02 · factories in P04-07**

📍

Anchor paths to the file

Build paths from `__file__`, never from the current directory.

When: every time you open a data file. It is the number-one CI failure for file-based data.

```
DATA = Path(__file__).resolve().parent
/ "data"
```

POWERS → **P02-03**



Decorators & closures

Wrap a function to add behaviour before or after it.

When: timing, retry, logging, skipping in certain environments.

```
@retry(times=3, exceptions=
(ConnectionError,))
def get_health(): ...
```

POWERS → **P02-05**



Context managers

Setup and teardown around a block — teardown is guaranteed.

When: files, timers, transactions, temporary overrides, `pytest.raises`.

```
with timer("checkout flow"):
    place_order()             #
teardown always runs
```

POWERS → **P02-03 · P02-05**



Generators

`yield` hands out values one at a time — constant memory, can stop early.

When: paginated APIs, log lines, unique test data — and every pytest `yield` fixture.

```
def iter_products(session, base_url):
    ...
    yield from body["data"]   # one
page at a time
```

POWERS → **P02-07 · fixtures in P03-04**



Hashing

A dict or set answers "how many?" and "seen before?" in O(1).

When: counting, deduplicating, grouping, finding a complement.

```
counts = Counter(s)           #
frequencies
seen: dict[int, int] = {}     # value
→ index
```

POWERS → **drills 3, 4, 5, 6, 13, 14, 20**

👉👈

Two pointers

Two indexes moving through the data instead of a nested loop.

When: comparing both ends (palindrome) or walking two sorted lists (merge).

```
i, j = 0, len(s) - 1
while i < j:                    # walk
inward from both ends
    i, j = i + 1, j - 1
```

POWERS → **drills 2, 11**

📚

Stack

Last in, first out — a list with `append` and `pop`.

When: things must close in the reverse order they opened: brackets, tags, nested calls.

```
stack.append(ch)                #
opener: push
stack.pop() ≠ pairs[ch]         #
closer: must match
```

POWERS → **drill 12**

🪟

Sliding window

Grow a window to the right; shrink it from the left when a rule breaks.

When: "longest / shortest contiguous run that satisfies…".

```
if last_seen.get(ch, -1) ≥ start:
    start = last_seen[ch] + 1   #
shrink past the repeat
```

POWERS → **drill 15**

🌳

Recursion

Solve the nested part by calling yourself on it.

When: tree-shaped data — nested lists, JSON, nested overrides.

```
diffs += json_diff(expected[key],
actual[key],
                   f"{path}.{key}")
```

POWERS → **drills 9, 18 · deep-merge in P02-02**

P02-01 · 🟢 ZERO · 3 H

Data types, collections & comprehensions

App — SauceDemo

Deliverable — `python_basics/products.py`

📋 The task

1. Log in to SauceDemo as `standard_user` and copy the 6 products (name and price) into a list of dicts in `products.py`.
2. Write functions that mirror the UI's sort dropdown: `sort_products(products, mode)` for `az`, `za`, `lohi` and `hilo`.
3. Write: `under(products, max_price)`, `cheapest(products)`, `total(products)`, `names_by_price(products) → dict[price, list[name]]`, `duplicate_prices(products)`.
4. Write `is_sorted(values, reverse=False) → bool`. You'll reuse it in P05 to verify the UI sort.

`done when` Tick these off

- [ ] All functions have asserts that pass (`python python_basics/products.py`)
- [ ] Prices use `Decimal`, not `float` (you can explain why)

📘 User guide

`concepts` What you need to know

The four collection types are in the first primer. Two more ideas carry this task:

Comprehension

`[expr for x in items if cond]` — a readable way to map and filter in one line.

`sorted()` vs `list.sort()`

`sorted(items, key=..., reverse=...)` returns a **new** list. `list.sort()` sorts in place and returns `None` (a classic bug).

`deep dive` Four things that separate a pass from a strong answer

References and mutability

`a = [1, 2]; b = a; b.append(3)` changes `a` too — both names point at the same list. Tests that share a mutable "default payload" dict and modify it leak state into each other. Copy first, or build fresh data per test.

Complexity you'll be asked about

`x in list` checks every element (O(n)); `x in set` / `x in dict` is a hash lookup (O(1) on average). Comparing expected vs actual IDs with sets (`expected - actual`) is both faster and clearer than nested loops.

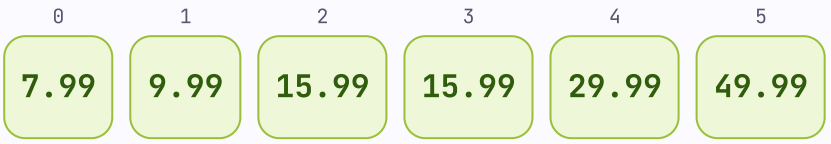
Money is not a float

`0.1 + 0.2 == 0.30000000000000004`. Use `Decimal("29.99")` (from a *string*) for prices and totals, otherwise your assertion that tax equals 8% will fail by a cent. It returns in P02-02 with rounding.

Check properties, not hard-coded lists

To verify "price low→high", don't hard-code the expected order: ties (SauceDemo has two $15.99 items) may appear either way round. Check the **property**: `prices == sorted(prices)`. Tests stay robust when the data changes.

INTERACTIVE — IS_SORTED() WALKS NEIGHBOUR PAIRS



Every pair passed → **is_sorted(values) is True** — with no hard-coded expected list.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 **Low: filter with a comprehension.** Six products as a list of dicts, and one line that keeps the names under $20.

```python
from decimal import Decimal

PRODUCTS = [
    {"name": "Sauce Labs Backpack", "price": Decimal("29.99")},
    {"name": "Sauce Labs Bike Light", "price": Decimal("9.99")},
    {"name": "Sauce Labs Bolt T-Shirt", "price": Decimal("15.99")},
    {"name": "Sauce Labs Fleece Jacket", "price": Decimal("49.99")},
    {"name": "Sauce Labs Onesie", "price": Decimal("7.99")},
    {"name": "Test.allTheThings() T-Shirt (Red)", "price": Decimal("15.99")},
]

cheap = [p["name"] for p in PRODUCTS if p["price"] < Decimal("20")]
assert "Sauce Labs Onesie" in cheap and len(cheap) == 4
```

Time O(n) **Space** O(k) results

SAY THIS

"One pass over the products, keeping names whose price is under twenty. Prices are Decimals built from strings, so the comparison is exact — no float surprises."

LEVEL 2 — IMPRESS THEM

🔵 **Medium: sort, group and find duplicates.** The dropdown's four modes become a dict of `(key, reverse)` pairs.

```python
from collections import Counter, defaultdict

def sort_products(products, mode: str):
    keys = {
        "az":   (lambda p: p["name"], False),
        "za":   (lambda p: p["name"], True),
        "lohi": (lambda p: p["price"], False),
        "hilo": (lambda p: p["price"], True),
    }
    key, reverse = keys[mode]            # KeyError for unknown mode = fail fast
    return sorted(products, key=key, reverse=reverse)

def names_by_price(products):
    grouped = defaultdict(list)
    for p in products:
        grouped[p["price"]].append(p["name"])
    return dict(grouped)

def duplicate_prices(products):
    counts = Counter(p["price"] for p in products)
    return {price for price, n in counts.items() if n > 1}

assert sort_products(PRODUCTS, "lohi")[0]["name"] == "Sauce Labs Onesie"
assert duplicate_prices(PRODUCTS) == {Decimal("15.99")}
```

Sort O(n log n) **Group / count** O(n) **Space** O(n)

HIRE SIGNAL

"A dict lookup replaces an if/elif ladder, and an unknown mode raises KeyError straight away — fail fast. `defaultdict` *and* `Counter` *group and count in a single pass."*

LEVEL 3 — HERO MODE

🟣 **High/Complex: property-based sort check with diagnostics (reused in P05).** Don't just say "not sorted" — say *where*.

```
def is_sorted(values, reverse: bool = False) -> bool:
    return all((a ⩾ b) if reverse else (a ⩽ b) for a, b in zip(values, values[1:]))

def first_unsorted_pair(values, reverse: bool = False):
    """Return (index, left, right) of the first violation, or None. Great for assertion
messages."""
    for i, (a, b) in enumerate(zip(values, values[1:])):
        if (a < b) if reverse else (a > b):
            return i, a, b
    return None

ui_prices = [Decimal("7.99"), Decimal("15.99"), Decimal("9.99")]   # e.g. what problem_user
shows
assert not is_sorted(ui_prices)
assert first_unsorted_pair(ui_prices) == (1, Decimal("15.99"), Decimal("9.99"))
```

Time O(n) | **Space** O(n) for the slice | **stops** at the first violation

WHY IT'S HERO-LEVEL

`zip(values, values[1:])` pairs each element with its neighbour — an idiom worth memorising. `first_unsorted_pair` returns the index and both values: exactly what a useful assertion message needs.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

z **List vs tuple?**

Both are ordered sequences. Lists are mutable and tuples are immutable. Use tuples for fixed records (e.g. `("standard_user", "secret_sauce")` in parametrize) and as dict keys, since they're hashable if their contents are. Use lists for collections that grow or change.

Z **Are dicts ordered in Python?**

Yes. Since Python 3.7, dicts preserve insertion order as a language guarantee. Don't rely on JSON *API responses* keeping key order for equality checks, though. Compare dicts, not strings.

H **Why is `x in my_set` faster than `x in my_list`?**

Sets and dicts are hash tables: membership hashes `x` and jumps to its bucket, O(1) on average. Lists scan every element, O(n). With 10k IDs to reconcile, that's the difference between milliseconds and seconds. Set operations (`-` , `&` , `|`) also express "missing/extra records" clearly.

H **Shallow vs deep copy? Give a testing example.**

A shallow copy (`dict(d)` , `list(l)` , `copy.copy`) copies the outer container but shares nested objects. A deep copy (`copy.deepcopy`) copies everything recursively. Example: a base booking payload with nested `bookingdates` . Shallow-copying it and changing `bookingdates["checkin"]` mutates the base for every other test. Use deepcopy or a factory function that builds fresh data.

H **Why shouldn't you use float for prices?**

Binary floats can't represent most decimals exactly (`0.1 + 0.2 ≠ 0.3`), so totals drift by fractions of a cent and equality assertions fail. Use `Decimal` constructed from strings, and apply explicit rounding rules (`quantize` with `ROUND_HALF_UP`) that match the business requirement.

🤖 AI ASSIST

After writing your functions, ask AI: *"Review for edge cases: empty list, ties, unknown sort mode."* Then add asserts for each edge case it finds.

P02-02 · ZERO · 3 H

Functions, type hints & exceptions

App — SauceDemo checkout

Deliverable — `python_basics/checkout.py`

The task

1. `calculate_totals(prices: list[Decimal], tax_rate: Decimal = Decimal("0.08")) → dict` returning `item_total`, `tax` and `total`, rounded to cents the way SauceDemo does. Verify against the UI by buying the Backpack ($29.99): the overview shows the tax and total.
2. `validate_checkout(first: str, last: str, postal: str) → None` raising a custom `CheckoutValidationError` with SauceDemo's exact messages (e.g. `Error: First Name is required`). Check the UI for the others.
3. `build_booking(**overrides) → dict` returning a valid Restful-Booker payload with sensible defaults that any field can override.

`done when` Tick these off

- [ ] Your totals match the SauceDemo UI for 1, 2 and 3 items
- [ ] Every function has type hints and a docstring
- [ ] Invalid input raises your custom exception with the right message

User guide

`concepts` What you need to know

`Parameters`

Positional, keyword, default values, `*args` (extra positional → tuple), `**kwargs` (extra keyword → dict).

`Type hints`

`def f(x: int) → str:`. They're documentation that tools (mypy, IDEs) check — they **aren't enforced at runtime**.

Exceptions

`raise`, `try/except/else/finally`, custom exception classes, `raise NewError(...) from original`.

deep dive The mutable default trap — a top interview question

Defaults are evaluated **once**, when the function is defined. A default `[]` is one list shared by every call. Use `cart: list | None = None` and create the list inside.

```
def add_item(item, cart=[]):          # the SAME list is reused across calls
    cart.append(item)
    return cart

print(add_item("Backpack"))           # ['Backpack']
print(add_item("Onesie"))             # ['Backpack', 'Onesie']   ← state leaked between
calls!

def add_item_fixed(item, cart: list | None = None):
    if cart is None:
        cart = []                     # a fresh list on every call
    cart.append(item)
    return cart

print(add_item_fixed("Backpack"))     # ['Backpack']
print(add_item_fixed("Onesie"))       # ['Onesie']
```

deep dive Rounding is a business rule

Python's `round(2.5) == 2` uses *banker's rounding* (round half to even), and `round(2.675, 2) == 2.67` because 2.675 is really 2.67499999… as a float. For money use `Decimal.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)` and confirm which rule the product uses. A one-cent mismatch is a real bug class.

INTERACTIVE — A SAUCEDEMO CART, PRICED TWO WAYS

Backpack · $29.99 | Bike Light · $9.99 | Bolt T-Shirt · $15.99

Fleece Jacket · $49.99 | Onesie · $7.99 | T-Shirt (Red) · $15.99

FLOAT · PLAIN + AND *

prices added with +	59.980000000000004
× 0.08	4.7984
total	64.7784

DECIMAL · CALCULATE_TOTALS()

item_total	59.98
× Decimal("0.08")	4.7984
tax · quantize, ROUND_HALF_UP	4.80
total	64.78

The float sum is already wrong before tax: 59.980000000000004. An == check against 59.98 fails. Decimal gives $64.78 every time.

deep dive

Exceptions as a design tool

Raise specific exceptions (`CheckoutValidationError`) rather than returning `False` or error strings. Callers can't accidentally ignore them, and pytest can assert them with `pytest.raises(..., match=...)` . Catch narrowly: `except KeyError` , never bare `except:` , which also swallows `KeyboardInterrupt` and hides bugs. Use `raise ... from e` to keep the original cause in the traceback.

deep dive

try / except / else / finally — the order

`try` runs; `except` handles failures; `else` runs only if there was no exception; `finally` **always** runs, which makes it the place for cleanup like closing a browser or deleting test data.

INTERACTIVE — WHICH BLOCKS RUN?

try	except KeyError	else	finally
the risky code	handle the failure	only if nothing raised	always — cleanup

Nothing was raised → **else** runs, then **finally**.

```
def run(step):
    try:
        print("try")
        step()
    except KeyError as e:
        print("except  → handles", repr(e))
    else:
        print("else    → only when nothing was raised")
    finally:
        print("finally → always (cleanup lives here)")

run(lambda: None)                # try, else, finally
print("---")
run(lambda: {}["missing"])       # try, except, finally
```

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 **Low: defaults and keyword arguments.** One parameter has a default; callers can pass arguments by name, in any order.

```
def greet(user: str, greeting: str = "Hello") → str:
    """Return a greeting for a user."""
    return f"{greeting}, {user}!"

assert greet("standard_user") == "Hello, standard_user!"
assert greet(greeting="Hi", user="problem_user") == "Hi, problem_user!"
```

concept default values | **concept** keyword arguments | **habit** hints + docstring

SAY THIS

"The greeting defaults to Hello, so the common call stays short. Passing by keyword makes the call site read like a sentence — and the type hints tell mypy and my IDE what goes in and what comes out."

LEVEL 2 — IMPRESS THEM

🔵 **Medium: money maths plus a custom exception.** The whole of task steps 1 and 2 in twenty lines.

```
from decimal import Decimal, ROUND_HALF_UP

CENT = Decimal("0.01")

class CheckoutValidationError(ValueError):
    """Raised when checkout info is invalid; message mirrors the UI."""

def calculate_totals(prices: list[Decimal], tax_rate: Decimal = Decimal("0.08")) →
dict[str, Decimal]:
    item_total = sum(prices, Decimal("0"))
    tax = (item_total * tax_rate).quantize(CENT, rounding=ROUND_HALF_UP)
    return {"item_total": item_total, "tax": tax, "total": item_total + tax}

def validate_checkout(first: str, last: str, postal: str) → None:
    for value, label in ((first, "First Name"), (last, "Last Name"), (postal, "Postal
Code")):
        if not value.strip():
            raise CheckoutValidationError(f"Error: {label} is required")

assert calculate_totals([Decimal("29.99")])["tax"] == Decimal("2.40")
try:
    validate_checkout("Ada", "", "12345")
except CheckoutValidationError as e:
    assert str(e) == "Error: Last Name is required"
```

Time O(n) prices | **rounding** ROUND_HALF_UP to the cent | **error** subclass of ValueError

HIRE SIGNAL

"I start the sum from `Decimal("0")` so the result is a Decimal even for an empty cart, I round once with an explicit mode, and I raise an exception whose message mirrors the UI — so one test can check both the function and the product."

LEVEL 3 — HERO MODE

🟣 **High/Complex: a payload builder with overrides, deep-merge and exception chaining.** This "defaults + overrides" builder is the core of good test-data design. Tests state *only what matters to them*.

```python
import copy
from typing import Any

DEFAULT_BOOKING: dict[str, Any] = {
    "firstname": "Test", "lastname": "User", "totalprice": 150, "depositpaid": True,
    "bookingdates": {"checkin": "2026-01-01", "checkout": "2026-01-05"},
    "additionalneeds": "Breakfast",
}

class PayloadError(Exception):
    pass

def _deep_merge(base: dict, overrides: dict) -> dict:
    for key, value in overrides.items():
        if isinstance(value, dict) and isinstance(base.get(key), dict):
            _deep_merge(base[key], value)
        else:
            base[key] = value
    return base

def build_booking(**overrides: Any) -> dict[str, Any]:
    payload = copy.deepcopy(DEFAULT_BOOKING)          # never mutate the default!
    unknown = set(overrides) - set(payload)
    if unknown:
        raise PayloadError(f"Unknown booking fields: {sorted(unknown)}")
    try:
        return _deep_merge(payload, overrides)
    except TypeError as e:
        raise PayloadError("Invalid override value") from e

b = build_booking(lastname="Smith", bookingdates={"checkout": "2026-01-10"})
assert b["bookingdates"] == {"checkin": "2026-01-01", "checkout": "2026-01-10"}
assert DEFAULT_BOOKING["lastname"] == "User"      # default untouched
```

deepcopy the default is never mutated

set difference rejects unknown fields

raise … from e keeps the cause

WHY IT'S HERO-LEVEL

Overriding only `checkout` keeps the default `checkin`: the merge recurses into nested dicts instead of replacing them. A typo such as `lastnam=` fails loudly instead of silently sending a wrong payload.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **What are `*args` and `**kwargs`?**

`*args` collects extra positional arguments into a tuple, and `**kwargs` collects extra keyword arguments into a dict. They're used for flexible APIs, such as a payload builder accepting any field overrides or a wrapper that passes arguments through to another function.

H **Explain the mutable default argument problem.**

Default values are evaluated once at function definition, so a default `[]` or `{}` is shared by every call and accumulates state. Fix: default to `None` and create a new object inside. In test code this causes "passes alone, fails in suite" bugs.

H **What does `round(2.5)` return, and why does it matter for testing?**

`2`. Python uses round-half-to-even (banker's rounding). Combined with float representation (`round(2.675, 2) == 2.67`), this means tests that compute expected money values with `round()` can disagree with a system that rounds half-up. Use `Decimal` with an explicit rounding mode that matches the requirement.

Z **Order of `try/except/else/finally`?**

`try` runs first. If it raises, the matching `except` runs. If it doesn't raise, `else` runs. `finally` always runs last, even after `return` or an unhandled exception, so it's used for cleanup.

SH **EAFP vs LBYL: which do you prefer in framework code?**

LBYL ("look before you leap", e.g. `if key in d`) checks first. EAFP ("easier to ask forgiveness than permission", i.e. `try: d[key] except KeyError`) attempts and handles failure. Python favours EAFP: it avoids race conditions (a file checked with `exists()` can vanish before you open it) and keeps the happy path clean. I use LBYL for cheap validation with a clear error message, and EAFP around I/O and external systems.

🤖 AI ASSIST

Ask AI *"why does round(2.675, 2) give 2.67?"*, then verify with `Decimal(2.675)` in a REPL — or in the money primer above, which runs exactly that.

P02-03 · ZERO · 2 H

Files & test-data formats

App — SauceDemo, Toolshop

Deliverable — `framework/data/saucedemo_users.csv`, `python_basics/data_io.py`

The task

1. Create `framework/data/saucedemo_users.csv` with columns `username,password,can_login,expected_error`, one row per SauceDemo user.
2. Create `framework/data/toolshop_users.json` with the Toolshop test users (see DEMO_APPS.md).
3. In `data_io.py`, write `load_csv(name)`, `load_json(name)` and `save_json(name, data)` that resolve paths **relative to the module**, not the current directory.
4. Read `BASE_URL` from an environment variable with a default.

done when — Tick these off

- [ ] Loading works from *any* working directory (`cd /tmp && python /path/to/data_io.py`)
- [ ] `can_login` is converted to a real `bool` (not the string `"False"`, which is truthy!)

User guide

concepts — What you need to know

`pathlib.Path`

Object-oriented paths: `Path(__file__).parent / "data" / "users.csv"`.

`with open(...) as f`

A context manager that **always** closes the file.

`CSV`

`csv.DictReader` gives one dict per row. **Everything is a string.**

`JSON and YAML`

JSON maps to dict / list / str / int / float / bool / None. YAML is a human-friendly superset of JSON (`yaml.safe_load`).

`Environment variables`

`os.environ.get("BASE_URL", "https://www.saucedemo.com")` — a value from outside the code, with a default.

deep dive The number-one CI failure for file-based data

`open("data/users.csv")` resolves against the *current working directory*, which differs between your IDE, the terminal, pytest's rootdir and a Docker container. Always anchor to the code file: `Path(__file__).resolve().parent`.

deep dive CSV gotcha: `bool("False") is True`

Any non-empty string is truthy. Convert explicitly: `row["can_login"].strip().lower() == "true"`. The example below reads CSV text from memory so it can run here; the reader behaves the same on a real file.

```
import csv, io

text = """username,password,can_login,expected_error
standard_user,secret_sauce,true,
locked_out_user,secret_sauce,false,"Epic sadface: Sorry, this user has been locked out."
"""
rows = list(csv.DictReader(io.StringIO(text)))    # same API as an open file

locked = rows[1]
print(repr(locked["can_login"]))       # 'false' - CSV gives you strings, always
print(bool(locked["can_login"]))       # True    ← any non-empty string is truthy!

for row in rows:
    row["can_login"] = row["can_login"].strip().lower() == "true"
print([(r["username"], r["can_login"]) for r in rows])
```

deep dive Choosing a format for test data

FORMAT	BEST FOR	WATCH OUT FOR
CSV	Tabular, data-driven cases edited by non-coders	No types, no nesting
JSON	API payloads, nested data	No comments
YAML	Environment configs, readable nested data	Indentation errors; use `safe_load` (plain `load` can construct arbitrary objects)
Python factories	Dynamic and unique data	Needs code to change

Static files suit *reference* data (users, expected messages). Dynamic factories (P04-07) suit *created* data (unique emails), which parallel runs need.

`task step 4` An environment variable with a default

```
import os

DEFAULT = "https://www.saucedemo.com"
print(os.environ.get("BASE_URL", DEFAULT))       # nothing set → the default

os.environ["BASE_URL"] = "http://localhost:8080"  # what CI or your shell would export
print(os.environ.get("BASE_URL", DEFAULT))       # the override wins

del os.environ["BASE_URL"]                        # leave the environment as we found it
```

🧪 Worked examples

These three read files from *your* repo, so they have no Run button — type them into `python_basics/data_io.py` and run them there.

LEVEL 1 — CRACK IT

🟢 **Low: read CSV rows.** Anchor the path, open the file, get a list of dicts.

python_basics/data_io.py

```python
import csv
from pathlib import Path

DATA = Path(__file__).resolve().parent.parent / "framework" / "data"

with (DATA / "saucedemo_users.csv").open(newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
assert rows[0]["username"] == "standard_user"
```

path anchored to `__file__`

newline="" lets the csv module handle line endings

encoding always explicit

SAY THIS

"I build the path from the module's own location, so it works from any directory. DictReader gives me one dict per row — and every value is a string until I convert it."

LEVEL 2 — IMPRESS THEM

🔵 **Medium: typed loading with correct booleans.** Convert once, in the loader, so tests receive real types.

python_basics/data_io.py

```python
import csv, json
from pathlib import Path

DATA = Path(__file__).resolve().parent.parent / "framework" / "data"

def load_csv(name: str) -> list[dict]:
    with (DATA / name).open(newline="", encoding="utf-8") as f:
        rows = list(csv.DictReader(f))
    for row in rows:
        if "can_login" in row:
            row["can_login"] = row["can_login"].strip().lower() == "true"
    return rows

def save_json(name: str, data: object) -> Path:
    path = DATA.parent.parent / "test-results" / name
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(data, indent=2, default=str), encoding="utf-8")
    return path
```

bool converted in one place

mkdir `parents=True, exist_ok=True`

default=str serialises Decimal, Path, dates

HIRE SIGNAL

"The loader owns the conversion, so no test can forget it. `save_json` *creates its folder if needed and returns the path, which makes it easy to attach the file to a report."*

LEVEL 3 — HERO MODE

🟣 **High/Complex: one loader for all formats, validated into dataclasses with precise errors.** Errors that name the **file and row** save hours when a teammate edits test data.

python_basics/data_io.py

```python
import csv, json
from dataclasses import dataclass, fields
from pathlib import Path
import yaml   # uv add pyyaml

DATA = Path(__file__).resolve().parent.parent / "framework" / "data"

@dataclass(frozen=True)
class SauceUser:
    username: str
    password: str
    can_login: bool
    expected_error: str = ""

def _read(path: Path) -> list[dict]:
    match path.suffix:
        case ".csv":
            with path.open(newline="", encoding="utf-8") as f:
                return list(csv.DictReader(f))
        case ".json":
            return json.loads(path.read_text(encoding="utf-8"))
        case ".yaml" | ".yml":
            return yaml.safe_load(path.read_text(encoding="utf-8"))
        case _:
            raise ValueError(f"Unsupported test-data format: {path.suffix}")

def load_as[T](name: str, model: type[T]) -> list[T]:          # Python 3.12 generic syntax
    path = DATA / name
    allowed = {f.name for f in fields(model)}
    out: list[T] = []
    for i, raw in enumerate(_read(path), start=1):
        extra = set(raw) - allowed
        if extra:
            raise ValueError(f"{path.name} row {i}: unknown columns {sorted(extra)}")
        if "can_login" in raw and isinstance(raw["can_login"], str):
            raw["can_login"] = raw["can_login"].lower() == "true"
        out.append(model(**raw))
    return out

users = load_as("saucedemo_users.csv", SauceUser)
```

match on the file suffix (3.10+)

def load_as[T] generic syntax (3.12+)

needs `uv add pyyaml`

WHY IT'S HERO-LEVEL

The dataclass is the schema: an unknown column fails with the file name and row number, and a missing required column fails when the model is built. `frozen=True` stops a test from editing shared reference data.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **Why use `with open(...)`?**

It guarantees the file is closed even if an exception happens mid-read, so there are no leaked handles or unflushed writes. It's the context-manager protocol (`__enter__` / `__exit__`).

H **Tests pass in the IDE but fail in CI with FileNotFoundError. Why?**

Almost always a path relative to the *current working directory*, which differs between runners. Fix: build paths from `Path(__file__).resolve().parent`, or a fixture that exposes the data dir, and never assume the CWD.

H **How do you choose between CSV, JSON, YAML and code for test data?**

CSV for flat data-driven tables that non-coders maintain. JSON for API-shaped nested payloads. YAML for per-environment config. Code factories for dynamic, unique data needed in parallel runs. Reference data is static; created data is generated. Validate whatever you load.

SH **How do you manage test data across dev, staging and pre-prod?**

Separate *config* (URLs, credentials via env and secrets) from *data*. Prefer tests that **create their own data via API** so they're environment-independent. Keep a minimal seeded reference set per environment (known users and products) versioned in the repo or seed scripts. Never depend on prod-like data you don't control, mask PII, and have a reset or cleanup strategy (see P06-05).

SH **Why `yaml.safe_load` instead of `yaml.load`?**

Full `load` can construct arbitrary Python objects from tags, which is a code-execution risk if the file is untrusted. `safe_load` only builds basic types. It's the same principle as never `eval`-ing test data.

🤖 AI ASSIST

Ask AI to generate 20 *edge-case* rows for your CSV (unicode names, whitespace, very long strings), then decide which are meaningful.

P02-04 · 🔵 HERO · 3 H

OOP & dataclasses

App — SauceDemo / Toolshop domain

Deliverable — `framework/models.py`, `python_basics/cart.py`

📋 The task

1. `@dataclass(frozen=True) class Product(name: str, price: Decimal)` and `@dataclass class User`.
2. `class Cart` with `add`, `remove`, `count`, `total` (reuse P02-02), `__len__` and `__contains__`.
3. `Product.from_dict()` as a `classmethod`.
4. Preview POM: an abstract `BasePage` (abstract `url` property, `open()`) and two subclasses, `SauceLoginPage` and `ToolshopLoginPage`, each with `login(user)`. They just `print` for now; Playwright arrives in P05.

done when Tick these off

- [ ] `Product` instances are hashable and usable in a set

- [ ] Instantiating `BasePage` directly raises `TypeError`
- [ ] You can explain composition vs inheritance with *your* classes

📘 User guide

`concepts` What you need to know

`Class / instance / self`

A blueprint, an object built from it, and the object's reference to itself.

`The four pillars`

Encapsulation (hide internals), **inheritance** (is-a), **polymorphism** (same interface, different behaviour), **abstraction** (expose what, hide how).

`@dataclass`

Auto-generates `__init__`, `__repr__` and `__eq__`. `frozen=True` makes it immutable and hashable.

`@property · @classmethod · @staticmethod`

A computed attribute · an alternative constructor · a namespaced helper.

`ABC`

`abc.ABC` plus `@abstractmethod` enforce that subclasses implement an interface.

`deep dive` Where OOP shows up in test frameworks

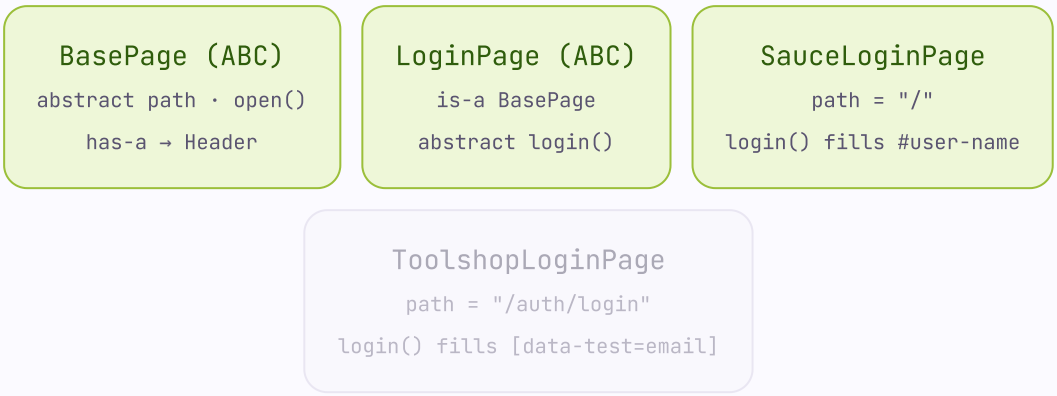
PATTERN	WHAT IT LOOKS LIKE	PILLAR
Page Object Model	each page is a class exposing *intent* (`login(user)`) while hiding locators	encapsulation
Components	a `Header` object *inside* several pages	composition

PATTERN	WHAT IT LOOKS LIKE	PILLAR
API clients	one class per resource (`BookingClient`) sharing a `BaseClient`	inheritance for shared plumbing
Models	dataclasses or pydantic models for payloads and responses	abstraction over raw dicts

deep dive Composition over inheritance

Deep hierarchies (`BasePage → AuthPage → AdminPage → AdminProductPage`) get brittle. Pages *have* components (header, cart badge, table) instead. Rule of thumb: inherit for **shared plumbing** (open, wait for load), compose for **shared UI pieces**.

INTERACTIVE — SMOKE_LOGIN(PAGE, CREDS): ONE TEST, MANY PAGES




```
page.open() → "https://www.saucedemo.com/" (inherited) · page.login(creds)
                    prints fill #user-name=standard_user
```

deep dive Dataclass vs pydantic

Dataclasses are lightweight and **don't validate types at runtime**. Pydantic validates and coerces (`"29.99"` → `Decimal`), which is ideal for **API responses** where you want the test to fail if the schema is wrong (P04-04).

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 **Low: dataclass plus a simple cart.** Two classes, one computed property.

```python
from dataclasses import dataclass, field
from decimal import Decimal

@dataclass(frozen=True)
class Product:
    name: str
    price: Decimal

@dataclass
class Cart:
    items: list[Product] = field(default_factory=list)   # never items: list = []

    def add(self, p: Product) -> None:
        self.items.append(p)

    @property
    def total(self) -> Decimal:
        return sum((p.price for p in self.items), Decimal("0"))

cart = Cart(); cart.add(Product("Onesie", Decimal("7.99")))
assert cart.total == Decimal("7.99")
```

frozen=True immutable + hashable

default_factory a fresh list per cart

@property `cart.total`, no brackets

SAY THIS

"A frozen dataclass gives me the constructor, a readable repr and equality for free. The cart's list uses `default_factory` — writing `items: list = []` would be the mutable-default trap again, shared by every cart."

LEVEL 2 — IMPRESS THEM

🔵 **Medium: alternative constructor and dunder methods.** `len(cart)` and `product in cart` just work.

```python
@dataclass(frozen=True)
class Product:
    name: str
    price: Decimal

    @classmethod
    def from_dict(cls, d: dict) -> "Product":
        return cls(name=d["name"], price=Decimal(str(d["price"])))

class Cart:
    def __init__(self) -> None:
        self._items: dict[Product, int] = {}          # encapsulated

    def add(self, p: Product, qty: int = 1) -> None:
        self._items[p] = self._items.get(p, 0) + qty   # works because Product is hashable

    def __len__(self) -> int:
        return sum(self._items.values())

    def __contains__(self, p: Product) -> bool:
        return p in self._items

# --- try it (added, not in P02) ---
onesie = Product.from_dict({"name": "Onesie", "price": 7.99})
cart = Cart()
cart.add(onesie, qty=2)
print(len(cart), onesie in cart)                        # 2 True  → __len__ and
__contains__
print({onesie, Product("Onesie", Decimal("7.99"))})     # ONE item: equal and hashable
```

`@classmethod` `from_dict` works for subclasses too

`_items` encapsulated

`__len__` · `__contains__`

HIRE SIGNAL

"`Decimal(str(d["price"]))` goes through a string, so a JSON float like 7.99 doesn't carry its binary error into my model. The cart can use products as dict keys only because the frozen dataclass is hashable."

LEVEL 3 — HERO MODE

🟣 **High/Complex: abstraction plus polymorphism — "same test, two apps".**

`smoke_login` knows nothing about SauceDemo or Toolshop; it only needs something that behaves like a `LoginPage`.

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass

@dataclass(frozen=True)
class Credentials:
    username: str
    password: str

class Header:                                        # a component (composition)
    def __init__(self, page_name: str) -> None:
        self.page_name = page_name
    def cart_count(self) -> int:
        return 0                                     # real locator in P05

class BasePage(ABC):
    def __init__(self, base_url: str) -> None:
        self.base_url = base_url
        self.header = Header(type(self).__name__)    # every page HAS a header

    @property
    @abstractmethod
    def path(self) -> str: ...

    def open(self) -> str:
        return f"{self.base_url}{self.path}"         # later: self.page.goto(...)

class LoginPage(BasePage, ABC):
    @abstractmethod
    def login(self, creds: Credentials) -> None: ...

class SauceLoginPage(LoginPage):
    path = "/"
    def login(self, creds: Credentials) -> None:
        print(f"fill #user-name={creds.username}")

class ToolshopLoginPage(LoginPage):
    path = "/auth/login"
    def login(self, creds: Credentials) -> None:
        print(f"fill [data-test=email]={creds.username}")

def smoke_login(page: LoginPage, creds: Credentials) -> None:   # polymorphism
    page.open(); page.login(creds)

smoke_login(SauceLoginPage("https://www.saucedemo.com"), Credentials("standard_user",
"secret_sauce"))

# --- try it (added, not in P02) ---
smoke_login(ToolshopLoginPage("https://practicesoftwaretesting.com"),
Credentials("customer2@practicesoftwaretesting.com", "welcome01"))
try:
    BasePage("https://example.com")                  # abstract → cannot be instantiated
except TypeError as e:
    print("TypeError:", e)
```

ABC can't be instantiated

composition every page *has* a Header

polymorphism one function, any LoginPage

WHY IT'S HERO-LEVEL

A plain class attribute (`path = "/"`) satisfies the abstract property. A third app is a new subclass — no existing test changes.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **Explain the four OOP pillars with test-automation examples.**

Encapsulation: page objects hide locators behind methods like `login()` . **Inheritance:** `BookingClient(BaseClient)` reuses session, auth and logging. **Polymorphism:** `smoke_login(page)` works with any `LoginPage` subclass. **Abstraction:** tests call `cart.checkout()` without knowing the clicks involved.

H **Composition vs inheritance: which do you prefer in a UI framework?**

Composition for UI pieces (header, table, modal components reused across pages). Inheritance only for thin shared plumbing (a `BasePage` with `open()` and wait-for-load). Deep hierarchies couple unrelated pages and make changes risky. "Has-a" is more flexible than "is-a".

H `@classmethod` vs `@staticmethod` ?

A classmethod receives the class (`cls`) and is used for alternative constructors like `Product.from_dict()` , which also works for subclasses. A staticmethod receives nothing; it's a plain function namespaced in the class for organisation.

H **Dataclass vs pydantic model vs plain dict for API responses?**

A dict is flexible, but typos surface late and there's no schema. A dataclass gives structure and a readable `repr` but no runtime validation. Pydantic gives structure plus runtime validation and coercion, so the test fails with a precise error if the API returns a wrong type or misses a field. For API tests I prefer pydantic, and dataclasses for internal test data.

SH **What is the MRO and when does it matter?**

Method Resolution Order is the order Python searches classes for an attribute with multiple inheritance (C3 linearisation, visible via `Cls.__mro__`). It matters for mixins (e.g. `LoggingMixin, BasePage`), where `super()` follows the MRO, not just the direct parent. Keep mixins small and stateless, or prefer composition to avoid surprises.

🤖 AI ASSIST

Ask AI to *critique* your class design: *"Where would this hierarchy become painful with 30 pages?"*

P02-05 · 🔵 HERO · 4 H

Decorators, context managers, logging & config

App — all (framework plumbing)

Deliverable — `framework/utils/retry.py`, `framework/utils/wait.py`, `framework/config.py`, `.env.example`

📋 The task

1. `@retry(times=3, delay=0.5, backoff=2.0, exceptions=(ConnectionError,))`, a decorator factory using `functools.wraps`.
2. `wait_until(condition, timeout=10, interval=0.25, message="")`, a poller that returns the condition's truthy value or raises `TimeoutError` with the message.
3. `timer(name)`, a context manager that logs the duration.
4. `framework/config.py`: a `Settings` class using **pydantic-settings** with fields for SauceDemo, Toolshop UI/API and Restful-Booker URLs, the default timeout and the environment name, loaded from `.env` with the prefix `QE_`.
5. Configure `logging` once (format: time, level, logger name, message).

done when Tick these off

- [ ] `QE_TOOLSHOP_API_URL=http://localhost:8091 python -c "from framework.config import settings; print(settings.toolshop_api_url)"` prints the override
- [ ] `.env` is git-ignored and `.env.example` documents every key
- [ ] `retry` only retries the listed exceptions, and `wait_until` returns early when the condition is met

📘 User guide

concepts What you need to know

Decorator

A function that takes a function and returns a wrapped one. `@d` above `def f` means `f = d(f)`. A **decorator factory** takes arguments and returns a decorator.

Closure

An inner function that remembers variables from its enclosing scope. It's how decorators keep `times`, `delay` and so on.

Context manager

Setup/teardown around a block (`with`). Built with `__enter__` / `__exit__` or `@contextlib.contextmanager`.

Logging

Levels `DEBUG < INFO < WARNING < ERROR < CRITICAL`; one logger per module (`logging.getLogger(__name__)`).

12-factor config

Config lives in the **environment**, not in code.

deep dive Why `functools.wraps` ?

Without it, every decorated function is named `wrapper` in tracebacks and reports, and pytest-style introspection breaks. `wraps` copies the name, docstring and signature

metadata.

deep dive Retry is a scalpel, not a bandage

Retrying *transient infrastructure* failures (connection reset, 503) on *idempotent* operations (GET, PUT) is reasonable. Retrying assertions or POSTs hides real bugs and creates duplicate data. Always cap attempts, back off exponentially, and log every retry so flakiness stays visible.

INTERACTIVE — @RETRY(DELAY=0.5, BACKOFF=2.0)

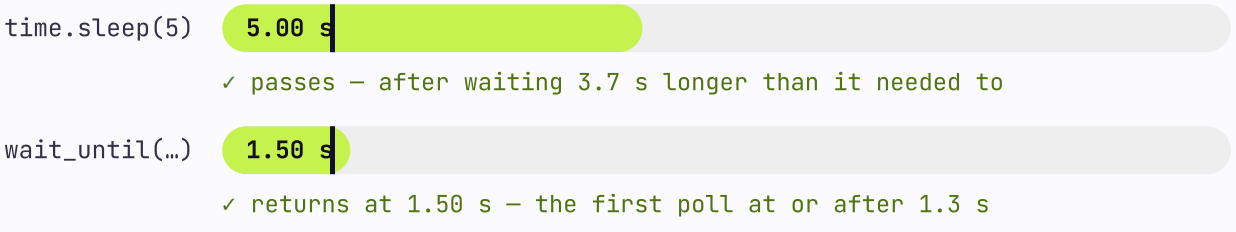


attempt 3: the call succeeds → **return** its value. Total backoff: 1.5s.

deep dive Polling vs sleeping

`sleep(5)` always costs 5 s, and fails if the system needs 6. `wait_until(cond, timeout=10)` returns as soon as `cond()` is true and fails with a clear message at 10 s. You'll use this for async states like order status in APIs and DBs. Playwright has its own auto-waiting for UI.

INTERACTIVE — DRAG TO CHANGE WHEN THE SYSTEM IS READY



The vertical line marks when the system becomes ready. Idealised: polls land at 0, 0.25, 0.50 … and the check itself takes no time.

deep dive Config precedence — highest wins

Real env vars → `.env` file → code defaults. CI sets env vars from secrets, developers use `.env`, and code defaults target the public demo apps. The same test code runs anywhere.

INTERACTIVE — WHAT DOES SETTINGS.TOOLSHOP_API_URL RESOLVE TO?

1 · real env var
e.g. exported by CI

2 · .env file
http://localhost:8091

3 · code default
https://api.practicesoftwaretesting.com

No env var, so the `.env` file wins → **http://localhost:8091**. Your laptop talks to the local stack.

deep dive

Logging vs print

Logs have levels, timestamps and sources. pytest captures them per test and shows them only on failure (or live with `log_cli = true`), and they can go to files. `print` has none of that. Task step 5 — configure it once:

```
import logging, sys

logging.basicConfig(                    # configure ONCE, at start-up
    level=logging.INFO, stream=sys.stdout, force=True,
    format="%(asctime)s %(levelname)-7s %(name)s: %(message)s",
)
log = logging.getLogger("framework.api.booker")    # in real code:
logging.getLogger(__name__)

log.debug("request headers: ...")       # hidden - below the INFO level
log.info("POST /booking → %s", 200)     # lazy %-formatting: only built if the line is
emitted
log.warning("retry %d/%d", 1, 3)
log.error("booking %s not found", 42)
```

Worked examples

The code above each `# --- try it` line is the P02 example, unchanged. The lines below it were added so that ▶ Run shows the behaviour.

LEVEL 1 — CRACK IT

Low: a timing decorator. The shape of every decorator you will ever write.

```python
import functools, logging, time

log = logging.getLogger(__name__)

def timed(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            log.info("%s took %.3fs", func.__name__, time.perf_counter() - start)
    return wrapper

# --- try it (added, not in P02) ---
import sys
logging.basicConfig(level=logging.INFO, stream=sys.stdout, force=True,
                    format="%(asctime)s %(levelname)s %(name)s: %(message)s")

@timed
def load_users():
    """Pretend to read the users file."""
    time.sleep(0.05)
    return ["standard_user", "problem_user"]

print(load_users())
print(load_users.__name__, "-", load_users.__doc__)    # preserved by functools.wraps
```

`*args, **kwargs` passes any call through

`finally` logs even when the call raises

`perf_counter` for durations

SAY THIS

"A decorator takes a function and returns a wrapper. `@timed` above `def f` is just `f = timed(f)`. I use `functools.wraps` so the wrapped function keeps its own name and docstring."

LEVEL 2 — IMPRESS THEM

🔵 **Medium: a retry decorator factory with backoff.** Three nested functions: the factory holds the settings, the decorator receives the function, the wrapper runs on every call.

```python
import functools, logging, time
from collections.abc import Callable

log = logging.getLogger(__name__)

def retry(times: int = 3, delay: float = 0.5, backoff: float = 2.0,
          exceptions: tuple[type[BaseException], ...] = (ConnectionError,)) -> Callable:
    def decorator(func: Callable) -> Callable:
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            wait = delay
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    if attempt == times:
                        raise
                    log.warning("%s failed (%s), retry %d/%d in %.1fs", func.__name__, e,
attempt, times, wait)
                    time.sleep(wait)          # sleep is fine HERE: it's backoff, not a
test wait
                    wait *= backoff
        return wrapper
    return decorator

# --- try it (added, not in P02) ---
import sys
logging.basicConfig(level=logging.INFO, stream=sys.stdout, force=True, format="%
(levelname)s %(message)s")
calls = {"n": 0}

@retry(times=3, delay=0.1)
def get_health() -> str:
    calls["n"] += 1
    if calls["n"] < 3:
        raise ConnectionError("connection reset")
    return "200 OK"

print(get_health(), "after", calls["n"], "attempts")

@retry(times=3, delay=0.1)
def create_order():
    raise ValueError("400 Bad Request")       # not in `exceptions` → never retried

try:
    create_order()
except ValueError as e:
    print("not retried:", e)
```

capped `times` attempts

backoff 0.5 → 1.0 → 2.0 s

narrow only the listed exceptions

HIRE SIGNAL

"On the last attempt I re-raise the original exception with a bare `raise`, so the traceback is untouched. Every retry is logged as a warning — a retry that hides flakiness is worse than no retry. The sleep here is backoff, not a test wait."

LEVEL 3 — HERO MODE

🟣 High/Complex: a poller, a timer context manager and typed settings.

framework/utils/wait.py

```python
# framework/utils/wait.py
import logging, time
from collections.abc import Callable, Iterator
from contextlib import contextmanager
from typing import TypeVar

T = TypeVar("T")
log = logging.getLogger(__name__)

def wait_until(condition: Callable[[], T], timeout: float = 10, interval: float = 0.25,
message: str = "") -> T:
    deadline = time.monotonic() + timeout            # monotonic: immune to clock changes
    last: T | None = None
    while time.monotonic() < deadline:
        last = condition()
        if last:
            return last
        time.sleep(interval)
    raise TimeoutError(f"Timed out after {timeout}s: {message or condition!r} (last value:
{last!r})")

@contextmanager
def timer(name: str) -> Iterator[None]:
    start = time.perf_counter()
    try:
        yield
    finally:
        log.info("%s: %.2fs", name, time.perf_counter() - start)

# --- try it (added, not in P02) ---
import sys
logging.basicConfig(level=logging.INFO, stream=sys.stdout, force=True, format="%
(levelname)s %(message)s")
started = time.monotonic()

def order_status() -> str | None:
    """Turns 'SHIPPED' after ~0.3 s, like an async backend job."""
    return "SHIPPED" if time.monotonic() - started > 0.3 else None

with timer("wait for order"):
    print(wait_until(order_status, timeout=2, interval=0.05, message="order never
shipped"))

try:
    wait_until(lambda: False, timeout=0.2, interval=0.05, message="stock never updated")
except TimeoutError as e:
    print("TimeoutError:", e)
```

time.monotonic immune to clock changes

returns the condition's value, not just True

@contextmanager setup · yield · teardown

The settings object and its example file. These need `pydantic-settings` and your repo, so run them there.

framework/config.py

```python
# framework/config.py      (uv add pydantic-settings)
from functools import lru_cache
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_prefix="QE_", extra="ignore")

    env: str = "public"
    saucedemo_url: str = "https://www.saucedemo.com"
    toolshop_url: str = "https://practicesoftwaretesting.com"
    toolshop_api_url: str = "https://api.practicesoftwaretesting.com"
    booker_url: str = "https://restful-booker.herokuapp.com"
    http_timeout: float = 10.0

@lru_cache
def get_settings() -> Settings:
    return Settings()

settings = get_settings()
```

.env.example

```
# .env.example: copy to .env (git-ignored) and adjust
QE_ENV=local
QE_TOOLSHOP_URL=http://localhost:4200
QE_TOOLSHOP_API_URL=http://localhost:8091
QE_BOOKER_URL=http://localhost:3001
```

WHY IT'S HERO-LEVEL

The timeout message includes the **last value seen**, so a failure tells you how far the system got. `@lru_cache` makes `get_settings()` build the settings once; every module then shares the same object.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

H **What is a decorator? Write one.**

A callable that takes a function and returns a new function that adds behaviour before or after it, e.g. timing, retry, logging, or skipping in certain envs. `@timed def f` is `f = timed(f)`. Write it with an inner `wrapper(*args, **kwargs)` and `functools.wraps(func)` to preserve metadata. (Then write the timing example from memory.)

H **Give examples of context managers in test automation.**

`with open(...)` for files; Playwright's `with page.expect_download()` and `with context.expect_page()`; `with pytest.raises(Error)`; DB transactions (`with engine.begin()`) that commit or roll back; a temporary env override; a timer. They guarantee teardown even when the block fails.

SH **When should you NOT retry?**

When the operation isn't idempotent (POST create or payment, where you'd risk duplicates), when the failure is deterministic (4xx validation, assertion failures), or when retries would mask a product bug or flaky test. Retry transient infra errors with caps, backoff and jitter, log every retry, and track retry rates as a health metric.

H **How do you manage configuration for multiple environments?**

A typed settings object that reads env vars, with a `.env` file for local development and defaults for the public environment. Precedence is env var > .env > defaults. Secrets come only from the environment or a secret store, never from committed files. The test code is environment-agnostic, and environment selection happens via one variable or a CLI option (`--env`, P03-05).

H **What is a closure?**

An inner function that captures variables from its enclosing scope and keeps them after the outer function returns. Decorator factories rely on it: `retry(times=3)` returns a decorator whose wrapper still "remembers" `times`.

🤖 AI ASSIST

Ask AI to write unit tests for `retry` using a fake function that fails N times. Check the tests don't actually sleep (hint: pass `delay=0` or monkeypatch `time.sleep`).

P02-06 · 🔵 HERO · 4 H

Coding interview drills

App — none | Deliverable — `python_basics/drills.py`

The task

Solve each of the 20 drills in **≤15 minutes**, talking out loud (record yourself once). Add asserts, including edge cases. The full list is the drill board below.

done when Tick these off

- [ ] 20 drills with asserts covering empty input, single element, duplicates and unicode where relevant
- [ ] You can state the time complexity of each solution

User guide

concepts What you need to know

Big-O

How cost grows with input size. O(1) hash lookup, O(n) single scan, O(n log n) sort, O(n²) nested loops.

Patterns

Hashing (dict/set for counts and seen-before), **two pointers** (palindrome, merge), **stack** (brackets), **sliding window** (longest substring), **recursion** (flatten, JSON diff).

deep dive A winning routine

QE coding rounds test **clear thinking and testing instinct** more than algorithms.

1. **Clarify:** input types, empty input, case sensitivity, duplicates, expected output format.
2. **Examples:** write 2–3 input→output pairs *including an edge case* before coding.
3. **Brute force first**, say its complexity, then improve if asked.
4. **Code cleanly**, naming things well and talking as you go.

5. **Test it yourself:** walk through your examples. As a QE, *listing edge cases unprompted* is where you shine.

The QA-flavoured drills (16–20) are increasingly common because they mirror real work: parsing logs, diffing API responses, validating data.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 **Low: first non-repeating character (drill 4).** Count everything once, then scan once more.

```python
from collections import Counter

def first_unique(s: str) -> str | None:
    counts = Counter(s)
    return next((ch for ch in s if counts[ch] == 1), None)

assert first_unique("swiss") == "w"
assert first_unique("aabb") is None
assert first_unique("") is None
```

`Time O(n)` `Space O(k) distinct chars` `pattern hashing`

SAY THIS

"Brute force is: for each character, count it in the whole string — O(n²). Counting once with a Counter makes it two passes, O(n). Edge cases: empty string and all-repeated both return None."

LEVEL 2 — IMPRESS THEM

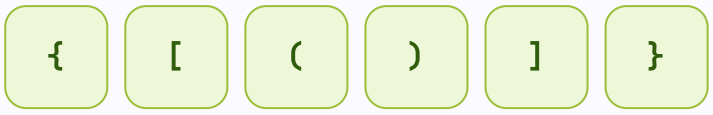
🔵 **Medium: valid brackets with a stack (drill 12).** Openers are pushed; every closer must match the most recent opener.

```python
def valid_brackets(s: str) -> bool:
    pairs = {")": "(", "]": "[", "}": "{"}
    stack: list[str] = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack

assert valid_brackets("{[()]}") and not valid_brackets("(]") and not valid_brackets("((")
and valid_brackets("")
```

Time O(n) **Space** O(n) **pattern** stack

INTERACTIVE — WATCH THE STACK



stack: empty

End of input and the stack is empty → **return True**

HIRE SIGNAL

"Two ways to fail: a closer with no matching opener, and openers left on the stack at the end — that's why the function returns `not stack` *, not just True."*

LEVEL 3 — HERO MODE

🟣 **High/Complex: recursive JSON diff with paths (drill 18) — QA gold.** It reports *where* two payloads differ, which is what a failing API assertion should tell you.

```python
from typing import Any

def json_diff(expected: Any, actual: Any, path: str = "$") -> list[str]:
    """Return human-readable differences, e.g. '$.bookingdates.checkin: expected 2026-01-
01, got 2026-01-02'."""
    if isinstance(expected, dict) and isinstance(actual, dict):
        diffs = []
        for key in expected.keys() | actual.keys():
            if key not in actual:
                diffs.append(f"{path}.{key}: missing in actual")
            elif key not in expected:
                diffs.append(f"{path}.{key}: unexpected field")
            else:
                diffs += json_diff(expected[key], actual[key], f"{path}.{key}")
        return sorted(diffs)
    if isinstance(expected, list) and isinstance(actual, list):
        if len(expected) != len(actual):
            return [f"{path}: length {len(expected)} != {len(actual)}"]
        return [d for i, (e, a) in enumerate(zip(expected, actual)) for d in json_diff(e,
a, f"{path}[{i}]")]
    return [] if expected == actual else [f"{path}: expected {expected!r}, got {actual!r}"]

exp = {"firstname": "Ada", "bookingdates": {"checkin": "2026-01-01"}, "tags": [1, 2]}
act = {"firstname": "Ada", "bookingdates": {"checkin": "2026-01-02"}, "tags": [1, 3],
"extra": True}
assert json_diff(exp, act) == [
    "$.bookingdates.checkin: expected '2026-01-01', got '2026-01-02'",
    "$.extra: unexpected field",
    "$.tags[1]: expected 2, got 3",
]
```

Time O(n) nodes | **Stack depth** = nesting depth | **pattern** recursion

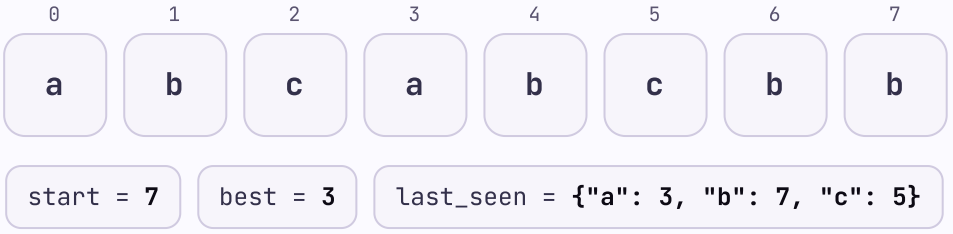
And the sliding window (drill 15). Grow the window to the right; when a character repeats inside it, jump the start past the earlier copy.

```python
def longest_unique(s: str) -> int:
    last_seen: dict[str, int] = {}
    start = best = 0
    for i, ch in enumerate(s):
        if last_seen.get(ch, -1) >= start:
            start = last_seen[ch] + 1          # shrink window past the repeat
        last_seen[ch] = i
        best = max(best, i - start + 1)
    return best

assert longest_unique("abcabcbb") == 3 and longest_unique("") == 0 and
longest_unique("bbbb") == 1
```

Time O(n) | **Space** O(k) distinct chars | **pattern** sliding window

INTERACTIVE — THE WINDOW NEVER MOVES LEFT



`start = 7` `best = 3` `last_seen = {"a": 3, "b": 7, "c": 5}`

Done → `longest_unique = 3`, in one pass.

🎯 The drill board — all 20

P02 lists the 20 drills and solves four of them. The other sixteen **reference solutions** here are `+ ADDED`. Set a 15-minute timer and solve the drill in your own file **first**; open the solution only to compare. Solved so far: **0 / 20**

01 Reverse a string (no slicing, then with)

`slicing`

Clarify first: is plain text enough, or must accented characters built from several code points survive? (Reversal works per code point.) **Complexity:** O(n) time, O(n) space.

```
def reverse_loop(s: str) → str:
    out: list[str] = []
    for i in range(len(s) - 1, -1, -1):
        out.append(s[i])
    return "".join(out)          # join once = O(n); `text += ch` in a loop can be
O(n^2)

def reverse_slice(s: str) → str:
    return s[::-1]

for reverse in (reverse_loop, reverse_slice):
    assert reverse("hello") == "olleh"
    assert reverse("") == "" and reverse("a") == "a" and reverse("añb") == "bña"
```

02 Palindrome (ignore case and non-alphanumerics)

two pointers

Clarify first: is an empty or punctuation-only string a palindrome? (Here: yes.)
Complexity: O(n) time, O(1) extra space — no cleaned copy of the string is built.

```python
def is_palindrome(s: str) -> bool:
    i, j = 0, len(s) - 1
    while i < j:
        if not s[i].isalnum():
            i += 1
        elif not s[j].isalnum():
            j -= 1
        elif s[i].casefold() != s[j].casefold():
            return False
        else:
            i, j = i + 1, j - 1
    return True

assert is_palindrome("A man, a plan, a canal: Panama")
assert is_palindrome("") and is_palindrome("?!") and is_palindrome("x")
assert not is_palindrome("race a car")
```

03 Character frequency

hashing

Clarify first: case-sensitive? Do spaces count? **Complexity:** O(n) time, O(k) space for k distinct characters.

```python
from collections import Counter

def char_frequency(s: str) -> dict[str, int]:
    counts: dict[str, int] = {}
    for ch in s:
        counts[ch] = counts.get(ch, 0) + 1
    return counts

assert char_frequency("hello") == {"h": 1, "e": 1, "l": 2, "o": 1}
assert char_frequency("") == {}
assert char_frequency("hello") == Counter("hello")      # the one-liner, once you've shown the loop
```

04 First non-repeating character

hashing

Solved by P02 — see **Worked examples → 🧠 Crack It** above. **Complexity:** O(n) time, O(k) space.

05 Anagram check

hashing

Clarify first: ignore case and spaces? (Here: yes, so "Dormitory" matches "dirty room".)
Complexity: O(n) time, O(k) space. `sorted(a) == sorted(b)` also works but is O(n log n).

```
from collections import Counter

def _letters(s: str) -> Counter[str]:
    return Counter(ch.casefold() for ch in s if ch.isalnum())

def is_anagram(a: str, b: str) -> bool:
    return _letters(a) == _letters(b)

assert is_anagram("Listen", "Silent") and is_anagram("Dormitory", "dirty room")
assert not is_anagram("abc", "abd") and not is_anagram("aab", "abb")
assert is_anagram("", "")
```

06 Remove duplicates, keep order

hashing

Clarify first: are the items hashable? (A list of dicts would need a key function.)
Complexity: O(n) time, O(n) space. `list(set(items))` is wrong here — it loses the order.

```
def dedupe(items: list) -> list:
    seen = set()
    out = []
    for x in items:
        if x not in seen:
            seen.add(x)
            out.append(x)
    return out

assert dedupe([3, 1, 3, 2, 1]) == [3, 1, 2]
assert dedupe([]) == [] and dedupe(["a"]) == ["a"]
assert dedupe([3, 1, 3, 2, 1]) == list(dict.fromkeys([3, 1, 3, 2, 1]))   # the
idiomatic one-liner
```

07 Second largest number

single scan

Clarify first: second largest *distinct* value? What comes back when there isn't one? (Here: `None`.) **Complexity:** O(n) time, O(1) space. `sorted(set(nums))[-2]` is shorter but O(n log n).

```
def second_largest(nums: list[int]) → int | None:
    """Second largest DISTINCT value, or None if there isn't one."""
    first = second = None
    for n in nums:
        if first is None or n > first:
            first, second = n, first
        elif n ≠ first and (second is None or n > second):
            second = n
    return second

assert second_largest([3, 9, 4, 9, 7]) == 7
assert second_largest([5, 5, 5]) is None and second_largest([]) is None and
second_largest([1]) is None
assert second_largest([-2, -1]) == -2
```

08 Missing number in 1..n

maths

Clarify first: exactly one number missing, and no duplicates? **Complexity:** O(n) time, O(1) space — the sum of 1..n minus the sum you were given.

```
def missing_number(nums: list[int]) → int:
    """nums holds n-1 distinct numbers from 1..n; return the one that is missing."""
    n = len(nums) + 1
    return n * (n + 1) // 2 - sum(nums)

assert missing_number([1, 2, 4, 5]) == 3
assert missing_number([]) == 1 and missing_number([2]) == 1 and missing_number([1])
== 2
```

09 Flatten a nested list

recursion

Clarify first: only lists nested, or tuples too? How deep can it go? **Complexity:** O(n) over all items. Recursion depth equals nesting depth — Python's default limit is about 1000.

```python
def flatten(items: list) -> list:
    out = []
    for x in items:
        if isinstance(x, list):
            out.extend(flatten(x))
        else:
            out.append(x)
    return out

assert flatten([1, [2, [3, [4]], 5], []]) == [1, 2, 3, 4, 5]
assert flatten([]) == []
assert flatten(["ab", ["cd"]]) == ["ab", "cd"]       # strings stay whole
```

10 FizzBuzz

loop

Clarify first: print or return? Is `n` included? **Complexity:** O(n). Building the word from two parts avoids a separate "divisible by 15" branch.

```python
def fizzbuzz(n: int) -> list[str]:
    out = []
    for i in range(1, n + 1):
        word = ("Fizz" if i % 3 == 0 else "") + ("Buzz" if i % 5 == 0 else "")
        out.append(word or str(i))
    return out

assert fizzbuzz(5) == ["1", "2", "Fizz", "4", "Buzz"]
assert fizzbuzz(15)[-1] == "FizzBuzz" and fizzbuzz(0) == []
```

11 Merge two sorted lists

two pointers

Clarify first: both ascending? Keep duplicates? (Here: yes.) **Complexity:** O(n + m) time and space. `sorted(a + b)` ignores that the inputs are already sorted.

```python
def merge_sorted(a: list[int], b: list[int]) -> list[int]:
    out: list[int] = []
    i = j = 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i])
            i += 1
        else:
            out.append(b[j])
            j += 1
    return out + a[i:] + b[j:]

assert merge_sorted([1, 4, 9], [2, 4, 10, 11]) == [1, 2, 4, 4, 9, 10, 11]
assert merge_sorted([], [1]) == [1] and merge_sorted([], []) == []
```

12 Valid brackets `()[]{}`

stack

Solved by P02 — see **Worked examples → 🚀 Impress Them** above, with the stack visualizer. **Complexity:** O(n) time, O(n) space.

13 Group anagrams

hashing

Clarify first: does output order matter? Case? **Complexity:** O(n · k log k) for n words of length k — each word is sorted once to make its key.

```python
from collections import defaultdict

def group_anagrams(words: list[str]) -> list[list[str]]:
    groups: dict[str, list[str]] = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w)      # sorted letters = the group's signature
    return list(groups.values())

assert group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]) == [
    ["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]
assert group_anagrams([]) == [] and group_anagrams([""]) == [[""]]
```

14 Two-sum (return indices)

`hashing`

Clarify first: is there always exactly one answer? May one element be used twice? (Here: no.) **Complexity:** O(n) time, O(n) space. The nested-loop brute force is O(n²).

```python
def two_sum(nums: list[int], target: int) -> tuple[int, int] | None:
    seen: dict[int, int] = {}                # value -> index
    for i, n in enumerate(nums):
        if target - n in seen:
            return seen[target - n], i
        seen[n] = i
    return None

assert two_sum([2, 7, 11, 15], 9) == (0, 1)
assert two_sum([3, 3], 6) == (0, 1)          # same value, two different indices
assert two_sum([1, 2], 7) is None and two_sum([], 0) is None
```

15 Longest substring without repeating chars

`sliding window`

Solved by P02 — see **Worked examples → 🏆 Hero Mode** above, with the window visualizer. **Complexity:** O(n) time, O(k) space.

16 Count ERROR lines per module in a log file

QA

Clarify first: the line format — this solution assumes `date time LEVEL module: message` — and what to do with malformed lines (skipped here). **Complexity:** O(n) lines; memory is O(modules) if you pass the open file instead of reading it all.

```
from collections import Counter

LOG = """\
2026-01-05 10:00:01 INFO checkout: order 17 created
2026-01-05 10:00:02 ERROR payment: card declined
2026-01-05 10:00:03 ERROR checkout: timeout calling payment
2026-01-05 10:00:04 WARNING payment: retrying
2026-01-05 10:00:05 ERROR payment: gateway 503
not a log line at all
"""

def errors_per_module(lines) -> dict[str, int]:
    counts: Counter[str] = Counter()
    for line in lines:                          # a list OR an open file - both are
read lazily
        parts = line.split(maxsplit=4)          # date, time, level, "module:",
message
        if len(parts) >= 4 and parts[2] == "ERROR":
            counts[parts[3].rstrip(":")] += 1
    return dict(counts)

assert errors_per_module(LOG.splitlines()) == {"payment": 2, "checkout": 1}
assert errors_per_module([]) == {}
```

17 Extract all emails from text (regex)

QA

Clarify first: how strict? This pattern is pragmatic, not a full RFC validator — say so. **Complexity:** O(n) over the text. New to `re`? See Beyond P02.

```
import re

EMAIL = re.compile(r"[\w.+-]+@[\w-]+(?:\.[\w-]+)+")

def extract_emails(text: str) -> list[str]:
    return EMAIL.findall(text)

text = "Contact ada@example.com, qe+run-1@mail.example.co.uk or call 555-0100. Not an
email: a@b"
assert extract_emails(text) == ["ada@example.com", "qe+run-1@mail.example.co.uk"]
assert extract_emails("") == []
```

18 Diff two JSON objects and report changed paths

QA

Solved by P02 — see **Worked examples → 🏆 Hero Mode** above. **Complexity:** O(n) nodes; recursion depth equals nesting depth.

19 Validate dates are `YYYY-MM-DD` and checkout > checkin

QA

Clarify first: may checkout equal checkin? (Here: no.) **Trap:** on Python 3.11+ `date.fromisoformat` also accepts `20260105`, and `strptime("%Y-%m-%d")` accepts `2026-1-5` — so check the shape first, then let `fromisoformat` reject impossible dates.

```
import re
from datetime import date

ISO_DATE = re.compile(r"[0-9]{4}-[0-9]{2}-[0-9]{2}")

def parse_date(value: str) -> date:
    if not isinstance(value, str) or not ISO_DATE.fullmatch(value):
        raise ValueError(f"{value!r} is not in YYYY-MM-DD format")
    return date.fromisoformat(value)          # also rejects impossible dates like
2026-02-30

def validate_stay(checkin: str, checkout: str) -> None:
    start, end = parse_date(checkin), parse_date(checkout)
    if end <= start:
        raise ValueError(f"checkout {checkout} must be after checkin {checkin}")

validate_stay("2026-01-01", "2026-01-05")     # valid -> no exception

bad_inputs = [("2026-01-05", "2026-01-01"), ("2026-01-01", "2026-01-01"),
              ("2026-1-5", "2026-01-09"), ("2026-02-30", "2026-03-01"),
("01/05/2026", "2026-01-09")]
for checkin, checkout in bad_inputs:
    try:
        validate_stay(checkin, checkout)
    except ValueError as e:
        print("rejected:", e)
    else:
        raise AssertionError(f"{checkin} -> {checkout} should have been rejected")
```

20 Find duplicate records by key in a list of dicts

QA

Clarify first: what about records where the key is missing? (Here: grouped under `None`.)
Complexity: O(n) time, O(n) space.

```python
from collections import defaultdict

def duplicates_by_key(records: list[dict], key: str) -> dict:
    """Map each duplicated key value to the records that share it."""
    groups = defaultdict(list)
    for rec in records:
        groups[rec.get(key)].append(rec)
    return {value: recs for value, recs in groups.items() if len(recs) > 1}

users = [
    {"id": 1, "email": "ada@example.com"},
    {"id": 2, "email": "bob@example.com"},
    {"id": 3, "email": "ada@example.com"},
    {"id": 4},                                  # key missing → grouped under None
]
dupes = duplicates_by_key(users, "email")
assert list(dupes) == ["ada@example.com"]
assert [u["id"] for u in dupes["ada@example.com"]] == [1, 3]
assert duplicates_by_key([], "email") == {}
```

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **How do you approach a coding question in an interview?**

Clarify requirements and edge cases, write examples, state a brute-force approach and its complexity, improve it if needed, code while explaining, then test with my examples and edge cases (empty, one element, duplicates, large input, invalid types).

H **What's the time complexity of two-sum with a dict, and why?**

O(n) time and O(n) space. One pass: for each number, check whether `target - n` is already in the dict (O(1) average lookup), then store `n → index`. Brute force is $O(n^2)$ with nested loops.

H **You wrote a function. Which test cases do you add?**

Happy path, boundaries (0, 1, max), empty or `None`, duplicates, negative numbers, unicode and whitespace for strings, very large input (performance), invalid types (expected exception), and idempotency or order-independence if relevant. Then equivalence partitions so I'm not writing redundant cases.

SH **Recursion vs iteration: what are the risks of recursion in Python?**

Python has no tail-call optimisation and a default recursion limit of about 1000, so deep inputs (a huge nested JSON) raise `RecursionError`. Recursion is clearest for tree-shaped data (JSON diff). For unbounded depth, use an explicit stack (iterative DFS).

🤖 AI ASSIST

Use AI as the **interviewer**, not the solver:

```
Give me a QA-flavoured Python coding problem, don't show the solution,
and review my code for edge cases after I paste it.
```

P02-07 · 🟣 SUPER HERO · 4 H

Advanced Python for framework authors

App — Toolshop API

Deliverable — `framework/api/pagination.py`, `framework/types.py`, framework passes `mypy --strict`

📋 The task

1. Write the generator `iter_products(session, base_url)`, which lazily yields every Toolshop product across all pages (`GET /products?page=N`, stopping at `last_page`).

2. Define `Protocol` interfaces for your clients, `TypedDict`s for request payloads, and an `Enum` for statuses.
3. Make `framework` an installable package (build system in `pyproject.toml`) and import it from tests without `sys.path` hacks.
4. Get `mypy --strict framework` to zero errors.

`done when` Tick these off

- [ ] `sum(1 for _ in iter_products(...))` equals the API's reported `total`
- [ ] `mypy --strict framework` passes
- [ ] `docs/python-advanced.md` explains Protocol vs ABC and generators vs lists in your own words

📘 User guide

`concepts` What you need to know

`Iterator / generator`

`yield` produces values lazily, one at a time. Memory stays constant however many items there are.

`typing.Protocol`

Structural typing ("anything with a `.get()` method fits") as opposed to ABC's nominal typing ("must inherit").

`TypedDict · Literal · Enum · TypeVar`

`TypedDict` gives type hints for dict *shapes* (JSON payloads); the rest name fixed choices and generics.

`Packaging`

`[build-system]` in `pyproject.toml` plus an editable install makes `import framework` work everywhere.

`deep dive` Generators are everywhere in pytest

A `yield` fixture *is* a generator: code before `yield` is setup, and code after is teardown. Understanding generators demystifies fixtures (P03-04). Here is what pytest does with one,

by hand:

```
def browser_fixture():
    print("setup:    launch browser")
    yield "<browser>"                 # the value pytest injects into the test
    print("teardown: close browser")

gen = browser_fixture()
browser = next(gen)                   # pytest runs the fixture up to `yield`...
print("test:     using", browser)
next(gen, None)                       # ...and resumes it after the test → teardown
```

deep dive Pagination as a generator

It hides page mechanics from tests. `for product in iter_products(...)` reads naturally, stops early if the consumer breaks, and never loads 10,000 products into memory.

INTERACTIVE — PAGES ARE FETCHED ONLY WHEN THE CONSUMER ASKS



`current_page ⩾ last_page → return`. The consumer counted 5 products — the API's total — holding one page in memory at a time.

The same thing in real Python, against a fake three-page API so it can run here + ADDED. Count the `[HTTP]` lines:

```python
from collections.abc import Iterator
from typing import Any

PAGES = {1: ["Pliers", "Bolt Cutters"], 2: ["Claw Hammer", "Thor Hammer"], 3: ["Hand Saw"]}

class FakeResponse:
    def __init__(self, body: dict[str, Any]) -> None:
        self.body = body
    def raise_for_status(self) -> None:
        return None
    def json(self) -> dict[str, Any]:
        return self.body

class FakeSession:
    """Stands in for requests.Session: 5 made-up products spread over 3 pages."""
    def get(self, url: str, params: dict[str, int], timeout: float) -> FakeResponse:
        page = params["page"]
        print(f"   [HTTP] GET {url}?page={page}")
        return FakeResponse({"data": PAGES[page], "current_page": page, "last_page":
len(PAGES), "total": 5})

def iter_products(session, base_url: str, timeout: float = 10) -> Iterator[Any]:
    page = 1
    while True:
        resp = session.get(f"{base_url}/products", params={"page": page}, timeout=timeout)
        resp.raise_for_status()
        body = resp.json()
        yield from body["data"]
        if body["current_page"] ⩾ body["last_page"]:
            return
        page += 1

print("count:", sum(1 for _ in iter_products(FakeSession(), "http://fake")))        # 3
requests
print("first:", next(p for p in iter_products(FakeSession(), "http://fake") if "Hammer" in
p))
# ^ only 2 requests: page 3 is never fetched, because the consumer stopped asking
```

deep dive Protocol vs ABC

	ABC	PROTOCOL
Typing	nominal — a class must *inherit*	structural — a class only needs the right methods
Checked	at instantiation (missing methods raise `TypeError`)	statically, by mypy
Shared code	yes — put real methods on the base	no — it is only a shape

	ABC	PROTOCOL
Good for	families of classes you *own*	third-party objects and fakes: both `requests.Session` and a test fake satisfy `HttpSession`

deep dive The GIL and parallel tests

CPython's Global Interpreter Lock lets only one thread execute Python bytecode at a time. Threads still help for I/O-bound work (HTTP calls release the GIL while waiting), but CPU-bound work needs processes. That's why `pytest-xdist` uses **worker processes** (P07). Newer Python versions offer an experimental free-threaded build, but test tooling still assumes processes.

deep dive Circular imports

They usually mean two modules need each other: `pages` importing `api` importing `pages`. Fix it by moving shared types to a leaf module (`framework/types.py`) or importing inside a function.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 **Low: an infinite generator of unique test emails.** It never ends — and never needs to.

```
import itertools, uuid
from collections.abc import Iterator

def unique_emails(prefix: str = "qe") -> Iterator[str]:
    run = uuid.uuid4().hex[:6]
    for n in itertools.count(1):
        yield f"{prefix}+{run}-{n}@example.com"

emails = unique_emails()
assert next(emails) != next(emails)
```

Memory O(1), however many you take

itertools.count counts forever

uuid unique per run

SAY THIS

"A generator function returns an iterator; each `next()` runs it up to the next `yield` and pauses it there. The run id makes emails unique across runs, the counter makes them unique within one — which parallel tests need."

LEVEL 2 — IMPRESS THEM

🔵 **Medium: a pagination generator over the Toolshop API.** This one calls the live API, so run it in your repo — the offline version in the user guide above runs here.

framework/api/pagination.py

```
# framework/api/pagination.py
from collections.abc import Iterator
from typing import Any
import requests

def iter_products(session: requests.Session, base_url: str, timeout: float = 10) →
Iterator[dict[str, Any]]:
    page = 1
    while True:
        resp = session.get(f"{base_url}/products", params={"page": page}, timeout=timeout)
        resp.raise_for_status()
        body = resp.json()                      # Laravel-style: data, current_page,
last_page, total
        yield from body["data"]
        if body["current_page"] ⩾ body["last_page"]:
            return
        page += 1

# usage: stop early without fetching every page
with requests.Session() as s:
    first_hammer = next(p for p in iter_products(s,
"https://api.practicesoftwaretesting.com")
                        if "hammer" in p["name"].lower())
```

`yield from` one page's items at a time

`timeout` on every HTTP call

`raise_for_status` fail fast on 4xx/5xx

HIRE SIGNAL

"Tests never see page numbers. If the API's pagination fields change, I update one function and every test keeps working — that is the point of the abstraction." Confirm the pagination field names in Swagger before you rely on them.

LEVEL 3 — HERO MODE

🟣 **High/Complex: Protocol, TypedDict and Enum for strict typing.** `FakeSession` never mentions `HttpSession` — it fits because it has the right methods.

framework/types.py

```python
# framework/types.py
from enum import StrEnum
from typing import Any, Protocol, TypedDict, NotRequired

class HttpResponse(Protocol):
    status_code: int
    def json(self) -> Any: ...
    def raise_for_status(self) -> None: ...

class HttpSession(Protocol):                    # requests.Session fits; so does a test fake
    def get(self, url: str, **kwargs: Any) -> HttpResponse: ...
    def post(self, url: str, **kwargs: Any) -> HttpResponse: ...

class BookingDates(TypedDict):
    checkin: str
    checkout: str

class BookingPayload(TypedDict):
    firstname: str
    lastname: str
    totalprice: int
    depositpaid: bool
    bookingdates: BookingDates
    additionalneeds: NotRequired[str]

class Role(StrEnum):
    ADMIN = "admin"
    CUSTOMER = "user"

class FakeSession:                              # structural match: no inheritance needed
    def __init__(self, payload: Any) -> None:
        self.payload = payload
    def get(self, url: str, **kwargs: Any) -> "FakeResponse":
        return FakeResponse(self.payload)
    def post(self, url: str, **kwargs: Any) -> "FakeResponse":
        return FakeResponse(self.payload)

class FakeResponse:
    status_code = 200
    def __init__(self, payload: Any) -> None:
        self.payload = payload
    def json(self) -> Any:
        return self.payload
    def raise_for_status(self) -> None:
        return None

# --- try it (added, not in P02) ---
def product_names(session: HttpSession, base_url: str) -> list[str]:
    resp = session.get(f"{base_url}/products")
    resp.raise_for_status()
    return [p["name"] for p in resp.json()["data"]]

fake = FakeSession({"data": [{"name": "Hammer"}, {"name": "Pliers"}]})
print(product_names(fake, "http://fake"))     # no network, no inheritance - the fake just fits
print(Role.ADMIN == "admin", list(Role))      # a StrEnum member IS a str
```

pyproject.toml

```
# pyproject.toml: make `framework` installable
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.hatch.build.targets.wheel]
packages = ["framework"]
```

Protocol structural interfaces

NotRequired optional payload keys

StrEnum members are strings

hatchling makes `framework` installable

WHY IT'S HERO-LEVEL

Functions accept the *protocol*, so unit tests pass a fake with no network and no inheritance, and `mypy --strict` proves the real client and the fake have the same shape.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

H **Generator vs list: when do you use each?**

A list holds all items in memory and can be iterated many times, indexed and measured with `len`. A generator produces items lazily, uses constant memory, can be infinite, and can be consumed only once. Use generators for large or unbounded streams (paginated APIs, log lines, unique data), and lists when you need random access or multiple passes.

H **How does a pytest `yield` fixture relate to generators?**

It *is* a generator function. pytest calls `next()` to run setup up to `yield` (the yielded value is injected into the test), and after the test it resumes the generator to run teardown. That's why teardown runs even if the test fails.

SH **Protocol vs ABC?**

ABCs use nominal typing (you must inherit), give you shared implementation, and fail at instantiation if methods are missing. Protocols use structural typing (any object with the right methods fits), checked statically by mypy. Protocols suit dependency injection and fakes; ABCs suit families of classes you own.

SH **What is the GIL and how does it affect parallel test execution?**

The GIL lets one thread run Python bytecode at a time per process. I/O-bound work (HTTP, browser I/O) still overlaps because waiting releases the GIL, but CPU-bound work doesn't parallelise with threads. pytest-xdist sidesteps this by running tests in separate **processes**, each with its own interpreter. That also isolates global state, so session fixtures run once *per worker*.

SH **How would you share a test framework across 10 repos?**

Package it (pyproject + build backend), version it with semver, publish to an internal index (or install from a Git tag), keep a changelog, and give it its own CI with unit tests. Keep the core small and stable (config, clients, base page, reporting hooks) and put app-specific pages and clients in each product repo. Deprecate before removing.

🤖 AI ASSIST

Paste mypy errors into AI and ask for an **explanation**, not a fix. Fix them yourself, since that's how typing clicks.

🏁 PHASE CHECKPOINT — P02

Prove it without help

⏱️ Live-coding self-test — 45 min, no AI

From an **empty file**, write these three, each with asserts:

- [ ] `calculate_totals` with Decimal rounding

- [ ] a `@retry` decorator
- [ ] `json_diff`

45-MINUTE TIMER

45:00

mock interview

AI mock interviewer prompt

Paste this into an AI assistant, answer out loud, and record your results in `interview-notes/P02.md`.

```
You are interviewing me for a QA Automation role (Python round). Ask 6 conceptual
questions (data types, mutability, decorators, generators, exceptions, OOP) and
then give me 2 coding problems of increasing difficulty, one QA-flavoured (logs/JSON).
Don't show solutions. After each answer: score 1-5 and one improvement tip.
```

BEYOND P02 — + ADDED

🧩 Ten gaps worth closing

P02 uses or assumes each of these, but never explains them: two drills need `re` and `datetime`, every file needs the `__main__` guard, and Python rounds for QE roles ask about the rest. One runnable example each, and why a QE needs it. (Strings, the eleventh gap, are in the primers.)

01 Identity, equality and truthiness

`==` asks "same value?"; `is` asks "the very same object?". And every object is either truthy or falsy — which is exactly why `bool("False")` bit you in P02-03.

```python
a = [1, 2]
b = [1, 2]
print(a == b, a is b)                # True False - equal values, two different objects

response = {"error": None, "items": [], "count": 0}
print(response["error"] is None)     # `is None` - the right test for None

for value in (None, 0, "", [], {}, "False", [0]):
    print(f"{value!r:>8} → {bool(value)}")
# `if not items:` can't tell "missing" from "empty" - assert the exact value you expect
```

Z `==` vs `is`?

`==` compares values (it calls `__eq__`); `is` compares identity — whether two names point at the very same object. Use `is` only for singletons: `None`, `True`, `False`. Two equal lists are `==` but not `is`.

02 Scope, closures and late binding

Python looks a name up in four places, in order: **L**ocal, **E**nclosing function, **G**lobal (module), **B**uilt-in. Closures (P02-05) capture the *variable*, not its value — which bites when you build checks or callbacks in a loop.

```python
checks = [lambda: f"check {i}" for i in range(3)]
print([c() for c in checks])            # ['check 2', 'check 2', 'check 2']   ← late binding

checks = [lambda i=i: f"check {i}" for i in range(3)]     # bind the value NOW
print([c() for c in checks])            # ['check 0', 'check 1', 'check 2']

def make_counter():
    count = 0
    def bump() -> int:
        nonlocal count                  # without this, `count += 1` raises UnboundLocalError
        count += 1
        return count
    return bump

attempt = make_counter()
print(attempt(), attempt(), attempt())  # 1 2 3 - the closure remembers `count`
```

H **Why do all my lambdas return the loop's last value?**

A closure looks its variable up when the function is *called*, not when it is defined (late binding). After the loop the variable holds its last value, so every lambda sees that. Bind the value at definition time with a default argument (`lambda i=i: ...`) or `functools.partial` .

03 Regular expressions

Drill 17 asks for a regex, and log parsing and message checks need them constantly. Five functions cover almost everything:

FUNCTION	USE IT TO
`re.search(p, s)`	find the first match anywhere; returns a match object or `None`
`re.fullmatch(p, s)`	**validate** — the whole string must match
`re.findall(p, s)`	get every match as a list
`re.sub(p, repl, s)`	replace — e.g. normalise whitespace before comparing UI text
`re.compile(p)`	name and reuse a pattern

```
import re

line = "2026-01-05 10:00:02 ERROR payment: card declined (order=17, amount=29.99)"

m = re.search(r"order=(\d+), amount=(\d+\.\d{2})", line)
print(m.group(1), m.group(2))                          # 17 29.99

pattern = re.compile(r"(?P<level>INFO|WARNING|ERROR) (?P<module>\w+):")
print(pattern.search(line).groupdict())                # {'level': 'ERROR', 'module':
'payment'}

print(re.findall(r"\d{4}-\d{2}-\d{2}", "from 2026-01-01 to 2026-01-05"))
print(bool(re.fullmatch(r"\d{5}", "12345")), bool(re.fullmatch(r"\d{5}", "12345-6789")))
print(re.sub(r"\s+", " ", "Sauce   Labs \n Backpack"))  # normalise whitespace before
comparing
```

H **`re.match` vs `re.search` vs `re.fullmatch`?**

`match` anchors at the start only, `search` finds the first match anywhere, and `fullmatch` requires the whole string to match — the right one for validation. Always write patterns as raw strings (`r"\d+"`) so backslashes reach the regex engine untouched.

04 Dates and times

Drill 19 validates booking dates, and Restful-Booker payloads carry them. Parse to real `date` objects before comparing — and keep timestamps **timezone-aware**, because your laptop and the CI runner are rarely in the same zone.

```
from datetime import date, datetime, timedelta, timezone

checkin = date.fromisoformat("2026-01-01")
checkout = checkin + timedelta(days=4)
print(checkout.isoformat(), (checkout - checkin).days)            # 2026-01-05 4

stamp = datetime.strptime("05/01/2026 14:30", "%d/%m/%Y %H:%M")   # parse a UI date format
print(stamp.strftime("%Y-%m-%dT%H:%M"))                           # 2026-01-05T14:30

created = datetime(2026, 1, 5, 23, 30, tzinfo=timezone.utc)
ist = timezone(timedelta(hours=5, minutes=30))
print(created.date(), "→", created.astimezone(ist).date())       # same instant, different
DAY

now = datetime.now(timezone.utc)          # timezone-aware; a naive datetime.now() depends
on the machine
print(now.tzinfo)
```

H **Naive vs aware datetimes — why does it matter in tests?**

A naive datetime has no `tzinfo` : it means "whatever this machine's local time is", so the same test can pass on a laptop and fail on a CI runner in UTC. An aware datetime carries its offset. Create timestamps with `datetime.now(timezone.utc)` , compare aware with aware, and convert only for display. (`datetime.utcnow()` returns a naive value and is deprecated since Python 3.12.)

05 The iteration toolkit

Small built-ins that replace index juggling. `enumerate` and `zip` already appear in P02's examples; `itertools.product` builds a test matrix, and `islice` takes a few items from a generator of any size.

```
from collections import deque, namedtuple
from itertools import islice, product

users = ["standard_user", "problem_user", "error_user"]
for i, name in enumerate(users, start=1):           # index + value, no range(len(...))
    print(i, name)

statuses = [200, 200, 503]
print(any(s ⩾ 500 for s in statuses), all(s == 200 for s in statuses))   # True False

first, *rest = users                                 # unpacking
print(first, rest)

print(list(product(["chromium", "firefox"], ["desktop", "mobile"])))     # a test matrix
print(list(islice((n * n for n in range(10**9)), 3)))                    # take 3 from a
lazy stream

recent = deque(maxlen=2)                             # keep only the last N log lines
for line in ["a", "b", "c"]:
    recent.append(line)
print(list(recent))

Row = namedtuple("Row", "username can_login")
print(Row("standard_user", True).username)
```

06 The `__main__` guard

P02 tells you to put your asserts under `if __name__ == "__main__":` — here is why. Run a file directly and Python names it `"__main__"`; import it and the name is the module's own. The guard keeps self-checks from running when P03's tests import your functions.

```
def total(prices: list[int]) → int:
    return sum(prices)

print("module name:", __name__)

if __name__ == "__main__":          # True when the file is run directly, False when it is
imported
    assert total([1, 2]) == 3
    assert total([]) == 0
    print("self-checks passed")
```

Z What does `if __name__ == "__main__":` do?

It runs the block only when the file is executed directly, not when it is imported. That lets one file be both a reusable module and a script with its own self-checks or command-line entry point.

07 The exception family tree

P02-02 says "never bare `except:`". The reason is the hierarchy: `KeyboardInterrupt` and `SystemExit` sit under `BaseException`, outside `Exception`. Note also that `ConnectionError` and `TimeoutError` are both kinds of `OSError` — useful when you choose what `@retry` should catch.

```
print(KeyboardInterrupt.__mro__[1].__name__, "←", "KeyboardInterrupt")   # BaseException
print(KeyError.__mro__[1].__name__, "←", "KeyError")                     # LookupError
print(issubclass(ConnectionError, OSError), issubclass(TimeoutError, OSError))   # True
True

class FrameworkError(Exception):
    """Base class: callers can catch everything the framework raises."""

class ApiError(FrameworkError):
    def __init__(self, status: int, body: str) -> None:
        super().__init__(f"API returned {status}: {body}")
        self.status = status

try:
    try:
        {}["token"]
    except KeyError as e:
        raise ApiError(401, "no token in response") from e    # keep the original cause
except FrameworkError as e:
    print(type(e).__name__, "-", e, "| caused by", repr(e.__cause__))
```

H Why is `except Exception` safer than a bare `except:`?

A bare `except:` catches `BaseException`, which includes `KeyboardInterrupt` and `SystemExit` — so Ctrl-C stops working and real bugs are swallowed. `except Exception` leaves those alone; catching the specific class is better still. A framework-wide base exception lets callers catch everything the framework raises in one clause.

08 Assertion messages that explain themselves

P02 checks everything with `assert`. Add a message, and format values with `!r` so that quotes, trailing spaces and `None` are visible in the failure.

```
expected, actual = "Sauce Labs Backpack", "Sauce Labs Backpack "

try:
    assert actual == expected, f"product name: expected {expected!r}, got {actual!r}"
except AssertionError as e:
    print("AssertionError:", e)     # !r prints the quotes, so the trailing space is
visible
```

09 Money through JSON

P02-01 says "money is not a float" — but `json.loads` turns every decimal number into one. Ask it for Decimals instead; `requests` passes the same argument through: `resp.json(parse_float=Decimal)`.

```
import json
from decimal import Decimal

body = '{"name": "Backpack", "price": 0.1, "tax": 0.2}'

as_float = json.loads(body)
print(as_float["price"] + as_float["tax"])              # 0.30000000000000004

as_decimal = json.loads(body, parse_float=Decimal)      # keep money exact straight off the
wire
print(as_decimal["price"] + as_decimal["tax"])          # 0.3

try:
    json.dumps(as_decimal)
except TypeError as e:
    print("TypeError:", e)                              # Decimal is not JSON serializable
print(json.dumps(as_decimal, default=str))              # the same trick save_json() uses
```

H **How do you keep prices exact when an API returns JSON numbers?**

Parse with `parse_float=Decimal` (or a pydantic model with `Decimal` fields) so the value never becomes a float. When writing JSON, serialise Decimals as strings with `default=str`.

10 async / await in one screen

This learning path uses Playwright's **sync** API (it matches pytest-playwright), but you will be asked about the async one. `async` code runs many waits on **one** thread: `await` hands control back while I/O is pending. This example has no Run button — `asyncio.run()` needs an ordinary Python process, so try it in your terminal.

```
import asyncio, time

async def fetch(name: str, seconds: float) -> str:
    await asyncio.sleep(seconds)          # hands control back while "waiting on I/O"
    return f"{name} done"

async def main() -> None:
    start = time.perf_counter()
    results = await asyncio.gather(fetch("users", 0.2), fetch("products", 0.2),
fetch("cart", 0.2))
    print(results, f"in {time.perf_counter() - start:.1f}s")    # ~0.2s, not 0.6s

asyncio.run(main())
```

H **Sync vs async Playwright — which and why?**

Both drive the same browser. The sync API reads top to bottom and fits pytest-playwright's fixtures. The async API uses `async` / `await` and suits code that already runs in an event loop or juggles many concurrent I/O tasks. Don't mix them: the sync API refuses to run inside a running event loop.

FIELD GUIDE

🎓 Pattern recognition

In the coding round: the signal in the question → the technique to reach for.

`count · most frequent · first unique`

Counter or dict

one pass, O(n)

`seen before? · duplicates`

set, or dict of value → index

O(1) membership

`a pair that adds up to a target`

dict of complements

O(n) instead of O(n²)

`must close in reverse order`

Stack

push openers, pop on closers

`longest / shortest contiguous run`

Sliding window

grow right, shrink left

`compare both ends · merge two sorted`

Two pointers

no nested loop

`nested data of unknown depth`

Recursion

or an explicit stack when it may be very deep

`is it sorted? where does it break?`

zip(values, values[1:])

check the property, not a fixed list

In your test suite: the symptom → the P02 fix.

`FileNotFoundError, but only in CI`

Path(__file__).resolve().parent

never rely on the current directory

`passes alone, fails in the suite`

Shared mutable data

None defaults, deepcopy, or a factory

`expected total is off by a cent`

Decimal from strings + quantize

and confirm the product's rounding rule

`a "false" from the CSV is treated as true`

Convert in the loader

every CSV value is a string

`every decorated function is named "wrapper"`

functools.wraps

keeps name, docstring, signature

`slow test that is still flaky`

wait_until, not sleep

returns early, fails with a message

`duplicate orders after a "flaky" run`

Don't retry non-idempotent calls

retry transient failures only, and log each one

`right URL locally, wrong one in CI`

Typed settings

env var > .env > default

`the result is None after sorting`

sorted() returns, .sort() doesn't

a classic bug

THE GOLDEN RULE

Clarify, write two examples, say the brute force and its complexity out loud — then code, and test it yourself. Listing edge cases unprompted is where a QE shines.

Built from *learning-path/phases/P02-python.md* · QE learning path, zero → hero

🐍 next: P03 — pytest, where these asserts become tests